

Forecast Bust Detection — Complete Team Handbook

SIH 2026 · Problem Statement SIH26079 · Ministry of Earth Sciences *AI-Based Forecast Bust Detection for Medium-Range Weather Forecasts*

Repository: github.com/blazingarrows1525/forecast-bust-detection

How to use this handbook

This is the single document for the whole team. It has four parts, ordered by what you need first.

Part	Read it when	Time
I — Learning path	You are new to the project and need to understand it	~12 h over 5 days
II — Presentation & jury defence	The night before presenting	~1.5 h
III — Technical reference	A mentor asks <i>how</i> something works	Lookup
IV — Data policy	Someone asks where the data is	10 min

If you have one evening before presenting: read Part II §0 and §1, then Part II §15 (the twenty judge answers) out loud. That gets you to "can defend it". Part I gets you to "can extend it".

The one rule that matters: the code is the source of truth. If any number in this handbook disagrees with the codebase, the codebase wins — and tell the team so it gets corrected.

Part I — The Learning Path

Audience: a teammate who did not build this and needs to understand it deeply enough to defend it, extend it, or answer a hostile question without hesitating.

This is a curriculum, not a reference. Two other documents already exist and this one tells you *when* to read them:

Document	What it is	When you use it
<code>TEAMMATE_BRIEFING.md</code>	Pitch defence — headline numbers, judge Q&A	The night before presenting
<code>TECHNICAL_STUDY_GUIDE.md</code>	Reference — math, algorithms, code paths	Open beside the code, look things up
<code>STUDY_PLAN.md</code> (this)	Curriculum — ordered path with exercises	Start here, work through it

Total time: ~12 hours of focused work across 5 days for the core track. Role tracks add 3–5 hours each.

The single rule: do the *Break it* exercises. Reading about a causality guarantee teaches you nothing; deliberately breaking one and watching a test go red teaches you everything. Every session has one.

Table of contents

- Day 0 — Setup (45 min)
- Day 1 — The problem and why the framing wins (2 h)
- Day 2 — The bust label, the heart of the project (2.5 h)
- Day 3 — Features, causality, and the poison test (2.5 h)
- Day 4 — Model, calibration, baselines (2.5 h)
- Day 5 — Explainability, refusal, serving (2 h)
- Role tracks
- Mastery check — 25 questions
- Common misconceptions
- Teach-back exercises

Day 0 — Setup (45 min)

Goal: a working environment where every command in this plan runs.

Prerequisites you genuinely need

You do **not** need meteorology. You need: - Python basics (functions, classes, pandas DataFrames) - What a classifier is, and what "probability of class 1" means - Comfort with a terminal

You do **not** need: deep learning, GIS experience, or NWP knowledge. Everything meteorological is explained where it appears.

Do this

```
git clone https://github.com/blazingarrows1525/forecast-bust-detection
cd forecast-bust-detection
pip install -r requirements.txt
```

Set `PYTHONPATH` for your shell (every command below assumes it):

```
export PYTHONPATH=src          # macOS/Linux
```

```
set PYTHONPATH=src
```

Verify the environment:

```
python -m pytest tests/ -q
```

Expected: `68 passed`. If you get import errors, `PYTHONPATH` is not set.

Get the dashboard data (63 MB, checksum-verified):

```
python scripts/fetch_release_artifacts.py
```

Start the server:

```
python -m uvicorn fbd.api.app:app --app-dir src --port 8912
```

Open `http://localhost:8912/` and `http://localhost:8912/command.html`. **Leave this running in its own terminal for the whole week.**

Day 0 checkpoint

You can answer: "how many tests pass, and what are the two dashboard URLs?" — 68, and `/` plus `/command.html`.

Day 1 — The problem and why the framing wins (2 h)

Goal: explain in your own words what the project predicts and why it isn't a weather forecaster.

Read (40 min)

- `README.md` top to bottom
- `TEAMMATE_BRIEFING.md` §0, §1, §3
- `LOGIC.md` §1 and §2

Run (20 min)

```
python scripts/replay_demo.py
```

This narrates the June 2022 Assam & Meghalaya bust in the terminal. Watch what it prints — this is the story you will tell a judge.

Then open `http://localhost:8912/command.html` and click the tall red column over the northeast. Read the right panel.

Observe

Answer these from the screen, not from memory: - What probability did the model give Assam & Meghalaya at Day 4? - What did the ensemble-spread baseline give for the same cell? - What actually happened (forecast mm vs observed mm)?

The concept that takes longest to click

Most people first assume the system predicts **rainfall**. It does not. It predicts **whether someone else's rainfall forecast is wrong**. The input is a forecast; the output is a probability that *that forecast* fails.

Analogy that helps: it is not a weather forecaster, it is a **spell-checker for forecasts**. It never writes the sentence; it underlines the word that looks wrong.

Break it (20 min)

Open `http://localhost:8912/api/bulletin/ASSAM_MEGHALAYA?init_date=2022-06-14` in a browser. Find the JSON field `bust_probability`. Now find `forecast_rain_mm` and `observed_rain_mm`.

Question to sit with: the model produced `bust_probability` *without ever seeing* `observed_rain_mm`. Where in the pipeline does the observation enter, and why can it not enter earlier? (Answer on Day 2.)

Day 1 self-check

1. In one sentence, what does this system output?
2. Why is "we forecast rainfall better than IMD" a losing pitch in 2026?
3. What does "the ministry already did the inversion" mean?

ANSWERS

1. A calibrated probability, per subdivision per lead day, that the existing operational rainfall forecast for that cell will bust — plus a plain-language reason. 2. Because GraphCast/GenCast/Pangu already beat physics models on average scores; you would lose that comparison in one judge question. 3. The problem statement (SIH26079) itself asks for bust probability, not better forecasts — so answering it literally **is** the novel framing. No reframing risk.

Day 2 — The bust label, the heart of the project (2.5 h)

Goal: derive the three-condition bust definition from scratch and explain why each condition is necessary.

This is the most important day. Anyone can fit a classifier once labels exist. The labels are the research contribution.

Read (45 min)

- `src/fbd/labels/bust.py` — **the whole file**, docstring included. It is 156 lines.
- `TECHNICAL_STUDY_GUIDE.md` §2 (the derivation)

Run (30 min)

```
python -c "  
import sys; sys.path.insert(0,'src')  
from fbd.labels.bust import rain_category  
for mm in [0, 2.4, 2.5, 15.5, 15.6, 64.4, 64.5, 115.5, 204.5, 500]:  
    print(f'{mm:7.1f} mm -> category {rain_category([mm])[0]}')  
"
```

Now look at the real labelled data:

```
python -c "
import sys; sys.path.insert(0, 'src'); import pandas as pd
d = pd.read_parquet('data/processed/dataset.parquet')
print(d.groupby('split').bust.agg(['size', 'mean']).round(4))
print()
print('bust rate by lead day:')
print(d.groupby('lead_day').bust.mean().round(4))
"
```

Observe

The bust rate **rises monotonically with lead day** (about 2.6% at Day 1 to 5.8% at Day 10). That is physically correct — longer forecasts fail more — and it is *emergent*, not imposed. Nothing in the label definition mentions lead day.

Also note: **test-year bust rate (3.59%) is lower than train (4.07%)**. Sit with why that matters. If the P95 threshold were re-fitted per year, both would be pinned near 5% by construction.

The three conditions

Write them out by hand before reading further:

```
BUST = Magnitude ^ CategoryFlip ^ Significance

Magnitude:    |F - 0| ≥ max(10 mm, P95_train(subdivision, month))
CategoryFlip: Cat(F) ≠ Cat(0)          using IMD's 6 official intensity bands
Significance:  max(F, 0) ≥ 15.6 mm      at least one side is "moderate" or above
```

Break it (45 min) — the important exercise

Open `tests/test_core.py` and find `test_bust_requires_all_three_conditions`. Read it.

Now deliberately weaken the definition. In `src/fbd/labels/bust.py`, find the line that computes the final label and remove the significance condition:

```
# find this
df["bust"] = (magnitude & category_flip & significant).astype("int8")
# change to
df["bust"] = (magnitude & category_flip).astype("int8")
```

Then:

```
python -m pytest tests/test_core.py -q
```

Watch it fail. Read the failure message. Then rebuild the dataset and see what happens to the bust rate:

```
python scripts/build_dataset.py
```

Compare the new bust rate to 4.07%. **Then undo your change** (`git checkout src/fbd/labels/bust.py`) and rebuild:

```
git checkout src/fbd/labels/bust.py && python scripts/build_dataset.py
```

Day 2 self-check

1. Why is a percentile-only definition circular?
2. Why does category-flip alone fail?
3. Where is P95 fitted, and what breaks if you fit it on all years?
4. Why do we require `max(F, 0) ≥ 15.6` rather than `≥ 64.5` (heavy)?

ANSWERS

1. Taking the 95th percentile of `|error|` defines exactly 5% of rows as busts by construction. The "bust rate" then measures the definition, not the atmosphere. 2. It fires on trivial boundary straddles (15.5 vs 15.7 mm) and almost never fires in dry subdivisions whose whole record maxes out below the heavy threshold. 3. Training years only (2016–2020), in `'bust.error_thresholds()'`. Fitting on all years pins the held-out bust rate at 5% and makes the evaluation meaningless. 4. Because area-averaging over a 19,000–222,000 km² subdivision dilutes extremes. A subdivision-mean of 64.5 mm/day is exceptionally rare; 15.6 mm/day already represents widespread significant rain and maps to a real advisory decision.

Day 3 — Features, causality, and the poison test (2.5 h)

Goal: explain the two causality axioms and prove they hold by breaking one.

Read (40 min)

- `src/fbd/features/forecast.py` — module docstring plus `lagged_ensemble()`
- `src/fbd/features/era5.py` — **the module docstring especially** (the causality rule)
- `TECHNICAL_STUDY_GUIDE.md` §3 and §10

The two axioms

Axiom A — no valid-time state. All ERA5 atmospheric features are sampled at initialisation time `t₀`, never at the forecast's target day. The atmosphere on the verification day does not exist when the forecast is issued; using it would leak the answer.

Axiom B — lagged-ensemble causality. For lead `L`, the lagged ensemble uses only leads `≥ L`. A forecast at lead `L-1` verifying the same day was issued *later in time*.

Convince yourself of Axiom B with a diagram. Draw a timeline. Mark `t₀`. Mark the valid day `V`. Now mark which initialisations produce a forecast for `V`, and which of those already exist at `t₀`.

Run (30 min)

```
python -m pytest tests/test_core.py::test_lagged_ensemble_uses_only_leads_at_or_beyond_L -v
```

Then open that test and read it. It poisons leads 1–4 with the value 1000 and leaves leads 5–10 at 10, then asserts that leads 5–8 show zero spread — because their members are drawn only from `{L, L+1, L+2}`, all unpoisoned.

Break it (50 min) — the causality poison

In `src/fbd/features/forecast.py`, find `lagged_ensemble()` and locate:

```
cols = [lead_pos[l] for l in range(L, L + n_members) if l in lead_pos]
```

Change it to look **backwards** (the leak):

```
cols = [lead_pos[l] for l in range(L - n_members + 1, L + 1) if l in lead_pos]
```

Run:

```
python -m pytest tests/test_core.py -q
```

The poison test goes red. That is the guarantee working.

Now do the dangerous thing — see what the leak buys you:

```
git stash                # save your break
git stash pop            # reapply it
python scripts/build_dataset.py
python scripts/train_model.py
```

Look at the AUROC. **It will be higher.** This is the single most important lesson in the project: *a leak looks like success*. There is no error message, no crash — just a better number. The only thing standing between you and a fraudulent result is the test.

Undo:

```
git checkout src/fbd/features/forecast.py
python scripts/build_dataset.py && python scripts/train_model.py
```

Confirm AUROC returns to ~0.840.

Day 3 self-check

1. Why can't the model use the ERA5 analysis for the forecast's valid day?
2. For lead 5, which leads form the lagged ensemble, and why not lead 4?
3. What is the observable symptom of a causality leak?
4. Why does the lagged spread have **NaN** at Day 10?

ANSWERS

1. It doesn't exist yet when the forecast is issued. It would be a near-perfect predictor of whether the forecast busted — i.e. showing the model the answer. 2. Leads 5, 6, 7. Lead 4 verifying the same day was initialised one day *after* `t0`, so it is in the future relative to the prediction moment. 3. A *better* score with no error. Leaks never crash; they just inflate metrics. Only tests catch them. 4. The archive stops at 240 h, so leads 11 and 12 don't exist; Day 10 has only one member and spread is undefined. Disclosed in `DECISIONS.md` D-011, and it's why the headline is Day 3–7.

Day 4 — Model, calibration, baselines (2.5 h)

Goal: explain every hyperparameter, why isotonic calibration is mandatory, and why the baseline comparison is fair.

Read (40 min)

- `src/fbd/model/train.py` — the whole file
- `src/fbd/model/baselines.py`
- `TECHNICAL_STUDY_GUIDE.md` §12 and §16

The parameters that matter

Read them from the code, not from a slide (the docs have drifted; **code is truth**):

```
python -c "
import sys, joblib; sys.path.insert(0, 'src')
d = joblib.load('data/artifacts/bust_model.joblib')
for k, v in d['params'].items(): print(f'{k:20s} {v}')
print()
print('n_features:', len(d['features']))
print('calibrator:', type(d['calibrator']).__name__)
"
```

The two lines a judge will ask about:

- `scale_pos_weight ≈ 25` — busts are ~4% of rows. Without this the model learns "always say no bust" and scores 96% accuracy while being useless.
- `IsotonicRegression fitted on the 2021 validation year` — because `scale_pos_weight` makes raw scores systematically over-confident. An uncalibrated bust probability is *worse than no probability*, because a forecaster would act on it.

Run (40 min)

```
python scripts/train_model.py
```

Read the whole output. Three tables print: overall, decision band, per-lead. Then:

```
python scripts/ablation.py
```

This shows what each feature group contributes. Note that **regime features add almost nothing to AUROC** — they are kept for explainability and independent validation, not for skill. Be ready to say that honestly.

The fairness protocol — why it matters

Every baseline gets the **same isotonic calibration** as the model, fitted on the same validation year. Read `scripts/train_model.py:Calibrated` to see it.

Without this, a class-weighted logistic regression emits ~0.5 probabilities and scores a Brier of 0.19 while having a perfectly respectable AUROC of 0.72. Beating that on Brier would prove nothing — it would be an artefact of the comparison.

Break it (30 min)

Remove the calibration and see what happens to reliability. In `src/fbd/model/train.py:predict_proba`, force the raw path:

```
def predict_proba(self, df):
    return self.predict_raw(df)          # calibration bypassed
```

Then:

```
python scripts/train_model.py
```

Compare **ECE** (expected calibration error) before and after. It should jump from ~0.011 to something much larger. AUROC will barely move — **calibration changes probabilities, not ranking**. That distinction is worth internalising.

Undo: `git checkout src/fbd/model/train.py`

Day 4 self-check

1. What does `scale_pos_weight` do and why is it ~25 here?
2. Why isotonic rather than Platt scaling?
3. Why does calibration change Brier but barely change AUROC?
4. What is the honest margin over a real IFS ensemble, and why are there two numbers?

ANSWERS

1. It multiplies the gradient contribution of positive rows by ~25, effectively balancing a 4%/96% problem. Without it the model collapses to always predicting "no bust". 2. Platt fits a 2-parameter sigmoid and assumes the raw-score-to-probability relationship is sigmoid-shaped. `scale_pos_weight` warps scores in a way that violates that; isotonic makes no shape assumption. 3. AUROC is a *rank* statistic and isotonic calibration is *monotone*, so ordering is preserved. Brier measures squared probability error, which calibration directly targets. 4. +0.025 AUROC raw (0.832 vs 0.807) and +0.040 calibrated (0.832 vs 0.792). AUROC needs no calibration so the raw number is the honest AUROC comparison; the calibrated one is required for Brier/cost. Quote both — quoting only the friendlier invites a cherry-picking challenge (`DECISIONS.md` D-014).

Day 5 — Explainability, refusal, serving (2 h)

Goal: explain how a SHAP number becomes an English sentence, and why refusing to answer is a feature.

Read (35 min)

- `src/fbd/explain/reasons.py` — the `TEMPLATES` dict and the family dedup logic
- `src/fbd/ood/detector.py` — the whole file, it is 99 lines
- `TECHNICAL_STUDY_GUIDE.md` §13, §14, §15

Run (35 min)

```
python scripts/stress_test.py
```

Look for the OOD table. The number to remember: **refused rows have a 23.4% bust rate vs 3.4% for accepted rows — about 7× higher.** That is what makes the refusal *earned* rather than decoration.

Then explore the reason engine live:

```
curl "http://localhost:8912/api/bulletin/ASSAM_MEGHALAYA?init_date=2022-06-14" | python -m json.tool
```

Find `dominant_factors` for lead 4. Three sentences, three *different concept families* — never three ways of saying "it's moist".

The two engineering tricks worth understanding

Concept-family dedup. SHAP often ranks `tcwv`, `moisture_flux_850`, and `mcz_q850_z` as the top three — all meaning "moist". The panel would say the same thing three times. Each feature maps to a family, and at most one factor per family is reported.

Direction-aware templates. A SHAP contribution can be positive for *either* direction of a feature. A single fixed wording produces sentences like "westerly flow is strong (-0.9 m/s, near normal)". Every template is a `(label, high_variant, low_variant)` triple.

Break it (30 min)

Find the `FAMILY` dict in `src/fbd/explain/reasons.py`. Map three moisture features to three *different* families (e.g. change `"tcwv": "moisture"` to `"tcwv": "moisture_a"`). Then:

```
python scripts/generate_bulletins.py --splits test
```

Query the same Assam row again and read `dominant_factors`. You should now see redundant moisture sentences. **That is what the dedup prevents.**

Undo: `git checkout src/fbd/explain/reasons.py` and regenerate.

Day 5 self-check

1. Why TreeSHAP and not KernelSHAP?
2. What makes an OOD refusal "earned"?
3. Why report only positive SHAP contributions in the reason panel?
4. What does `confidence_in_estimate` measure — and what does it *not* measure?

ANSWERS

1. TreeSHAP is exact and runs in $O(T \cdot L \cdot D^2)$ — milliseconds for our model. KernelSHAP is model-agnostic but $O(2^n)$ in features, requiring Monte-Carlo approximation with variance. For trees, exact beats approximate. 2. Refused rows bust $\sim 7\times$ more often than accepted rows (23.4% vs 3.4%). If refused rows busted at the same rate, the detector would be refusing at random and should be deleted. 3. The panel answers "why is confidence *low* here". Mixing in reassuring factors would bury the signal. 4. It measures how much the probability would move if the model had seen a slightly different training history (bootstrap-bagged refits). It does *not* measure whether the underlying weather is predictable — that's the best probability itself.

Role tracks

After the 5-day core, pick the track matching what you'll own.

Track A — Data & Features (3 h)

Read `src/fbd/ingest/`, `src/fbd/regions/masks.py`, `src/fbd/features/era5.py`. Then `TECHNICAL_STUDY_GUIDE.md` §4, §5, §9, §11.

Key ideas to master: - Why exact polygon-cell overlap beats centroid masking (it silently drops Konkan & Goa — the highest-rainfall region) - The chunking economics: WeatherBench 2 zarr chunks span the globe, so subsetting to India saves memory but **not bandwidth** - Why the ERA5 store named `1959-2022` actually ends in 2021 (`DECISIONS.md` D-009)

Exercise: run `python -m fbd.regions.build` and confirm 641 districts partition into 36 subdivisions with no district assigned twice.

Track B — Modelling & Evaluation (4 h)

Read `src/fbd/evaluate/metrics.py` end to end. Then `TECHNICAL_STUDY_GUIDE.md` §17–§20.

Key ideas: - Why AUROC is class-balance invariant and why that matters at a 4% base rate - Murphy's Brier decomposition (reliability – resolution + uncertainty) - Equal-*count* reliability binning, not equal-width — with rare events, equal-width leaves upper bins nearly empty - The asymmetric cost: `C_miss = 10`, `C_false = 1`, and Richardson's economic value

Exercise: change the cost ratio in `src/fbd/config.py` from 10:1 to 3:1, re-run `train_model.py`, and observe how the optimal threshold and `value` shift.

Track C — Serving & Frontend (3 h)

Read `src/fbd/api/app.py`, `src/fbd/api/schema.py`, `web/index.html`, `web/command.html`. Then `TECHNICAL_STUDY_GUIDE.md` §21–§25.

Key ideas: - The Pydantic double-guard: request params *and* response model both validated - Two clock modes (`replay` vs `live`) and why live mode correctly reports STALE against a 2022 archive - The geometry-merge optimisation in the 3D view: 1,669 → 343 draw calls - Air-gap verification: every asset vendored, zero CDN

Exercise: add a new field to `BustPrediction` in `schema.py`, watch the API fail loudly until you populate it. That failure is the schema doing its job.

Track D — MLOps & Verification (3 h)

Read `src/fbd/mlops/drift.py`, `src/fbd/quality/`, `tests/` in full. Then `TECHNICAL_STUDY_GUIDE.md` §26, §27, and `DECISIONS.md` D-016, D-017.

Key ideas: - PSI (Population Stability Index) and what a PSI of 2.64 on `india_shear_z` means - Why the drift monitor saying RETRAIN on a year the model handled well is an *interesting* finding, not a bug - The verification ledger — what has and has not actually been run

Exercise: run `python scripts/monitor_drift.py` and explain each row of the output to another teammate.

Mastery check — 25 questions

Answer without looking. If you miss more than five, revisit the relevant day.

Problem framing 1. What does the system output, precisely? 2. Why is this not a weather forecasting model? 3. What did the ministry itself already do that removes our framing risk?

Labels 4. State all three bust conditions. 5. Why is percentile-only circular? 6. Where is P95 fitted? 7. Why is the test-year bust rate 3.59% and not 5%?

Data 8. Name the five datasets and their roles. 9. Why is IMD gauge data preferred over ERA5 for rainfall? 10. How many subdivisions are modelled, and why not 36? 11. What is the 3-hour offset problem?

Features 12. State both causality axioms. 13. For lead 5, which leads form the lagged ensemble? 14. What is the observable symptom of a leak? 15. Why does Day 10 have no lagged spread?

Model 16. What is `scale_pos_weight` and why ~25? 17. Why isotonic rather than Platt? 18. Why does calibration change Brier but not AUROC? 19. Name the four baselines. 20. What is the honest margin over real IFS ENS — both numbers?

Explainability & uncertainty 21. Why TreeSHAP over KernelSHAP? 22. What is concept-family dedup and why is it needed? 23. What makes the OOD refusal earned? 24. What does `confidence_in_estimate` actually measure?

Honesty 25. Name three things this project has *not* done.

ANSWER TO 25 (THE ONE PEOPLE FUMBLE)

No real AWS Bedrock invocation (account on Free Plan, GenAI layer tested only against a mock); Terraform never applied or even validated; no ECR/ECS deployment or public demo URL. All three are stated in `DECISIONS.md` D-017's verification ledger. Saying this confidently is a **strength** — it shows you distinguish what you ran from what you wrote.

Common misconceptions

Every one of these has been said by someone learning this project. Correct them early.

"It predicts rainfall." No. It predicts whether *someone else's* rainfall forecast will be wrong. The rainfall forecast is an input.

"Higher AUROC is always better." Not if it came from a leak. On Day 3 you will personally produce a *higher* AUROC by breaking causality. Score improvements must be explainable.

"The model uses observed rainfall as a feature." It does not, and cannot — observations don't exist at forecast time. `obs_rain_mm` appears in the dataset only to *construct the label*, never in the 52 features.

"We should have used deep learning." $\sim 10^5$ tabular rows, mandated explainability, and a 4% rare class. Trees win on all three. Being able to explain *why not* is worth more than using it.

"Regime features are the main contribution." Honestly, they add little AUROC (see `ablation.py`). They're kept for explainability and because they validate independently — depression days over Odisha show 25.1 mm/day vs 5.3. Say this honestly rather than overclaiming.

"The 96% accuracy is great." Never quote accuracy. At a 4% base rate, predicting "never bust" scores 96% and is worthless. This is why `LOGIC.md` §4.5 bans it.

"OOD refusal is us hedging." The opposite. Refused rows bust $7\times$ more often. Refusing is the *correct* action on states unlike anything in training, and it's what a safety-critical tool should do.

Teach-back exercises

The fastest way to find out whether you actually understand something is to explain it to someone who will interrupt.

Exercise 1 — the 60-second pitch. Explain the project to a teammate who knows nothing, in 60 seconds, without saying "machine learning" or "AUROC". If they can't repeat back what it does, try again.

Exercise 2 — the label defence. Have a teammate play a hostile reviewer. They ask: "*isn't your bust definition arbitrary?*" You have three minutes. You must explain all three conditions and why removing any one breaks it.

Exercise 3 — the leak demo. Live-demo the Day 3 poison test to a teammate. Break causality, show AUROC rise, show the test go red, restore. This is the single most persuasive thing you can show a technically literate reviewer.

Exercise 4 — the honest answer. A teammate asks: "*how do you know you didn't overfit the 2022 test year through your own iteration?*" Practice the honest answer (see `TECHNICAL_STUDY_GUIDE.md` §32). Do not bluff. The honest answer earns more respect than a confident false one.

Exercise 5 — the live demo, cold. Run the full dashboard demo — 3D command centre, click the red column, read reasons, hit Truth, switch to 2D, show the metrics table — in under four minutes without notes. Do it three times.

Quick reference — the numbers to have memorised

Fact	Value
Modelled subdivisions	34 (not 36 — islands excluded)
Dataset	279,650 rows × 93 columns
Features	52, in 6 concept families
Splits	train 2016–20 / val 2021 / test 2022
Bust rate (test)	3.59%
Model AUROC (all leads)	0.840
Model AUROC (Day 3–7)	0.825
vs real IFS ENS	+0.025 raw / +0.040 calibrated
Decision cost reduction	–16.8%
ECE	0.011
OOD earned refusal	23.4% vs 3.4% ($\approx 7\times$)
Tests	68
Decisions logged	D-001 ... D-018

If you only have one evening

Do this, in order (~2 hours):

1. `TEAMMATE_BRIEFING.md` §0 and §1 (10 min)
2. Day 1's demo: `replay_demo.py` + click through the 3D dashboard (20 min)
3. Day 2's reading: `src/fbd/labels/bust.py` (30 min) — **the single most important file**
4. Day 3's poison test: run it, read it (20 min)
5. `TEAMMATE_BRIEFING.md` §15 — the 20 judge answers, out loud (40 min)

That gets you to "can defend it". The full five days get you to "can extend it".

Part II — Presentation & Jury Defence

Read this file end-to-end before touching the deck. It replaces every other document. `LOGIC.md`, `DECISIONS.md`, `HANDOFF.md`, `SESSION_HANOVER.md`, `PROJECT_BLUEPRINT.md` and `README.md` are still authoritative for the code — this file is written for a human who was not in the build, needs to defend it in front of a jury, and must be able to answer any question in under 30 seconds.

Prepared 24 Aug 2026 for handover to the teammate presenting on 25 Aug 2026.

0. The two sentences you MUST be able to say cold

"We do not compete with IMD's forecast. We tell IMD's duty forecasters which forecasts to double-check."

"On the held-out 2022 monsoon, our model beats the real 50-member IFS ensemble by +0.040 AUROC and cuts asymmetric decision cost by 16.8%."

The first sentence is the pitch. The second is the number that survives a hostile judge.

If nothing else in this document lands, land those two.

1. Executive summary — one page

Problem statement: SIH 2026 · SIH26079 · Ministry of Earth Sciences · *AI-Based Forecast Bust Detection for Medium-Range Weather Forecasts*.

What "forecast bust" means: a case where the operational rainfall forecast for a subdivision-day is badly wrong in a way that would have changed an alert decision — for example, the model forecast 20 mm/day of light rain and the observed rainfall was 120 mm/day of "very heavy" with associated flooding.

Why this is worth doing: IMD's medium-range bulletins go out every morning covering Day 1 to Day 10 across 36 meteorological subdivisions. Every forecast on that map looks equally trustworthy. There is no reliability metadata attached. Officers cannot tell a quiet-week forecast apart from one issued during a low-predictability regime, so both false alarms (which burn public trust) and missed events (which cost lives) trace back to the same missing information layer.

Why now: three findings from 2023–2026 make forecast-bust prediction the right ML problem to attack: 1. AI weather models (GraphCast, Pangu, GenCast) now beat physics models on average scores. 2. **AI and physics models bust on the same days** (ECMWF: IFS and Pangu shared a bust; day-6 daily error correlation ~0.54). Adding more models does not save you because they fail together. 3. AI models are *worst* on record-breaking extremes (Science Advances 2026) — precisely the days that matter operationally.

Together these say: failure is driven by the atmospheric state, not by any one model's quirks, so it is in principle predictable from that state. That is the ML opportunity nobody has yet solved for India.

What we built: a calibrated meta-model that scores every (subdivision × lead day × init date) combination in the held-out 2022 monsoon with a probability that the operational forecast for that cell busts, an out-of-distribution refusal layer, a plain-language meteorological reason for every high-probability flag, a two-tier dashboard (2D choropleth + 3D command centre showing space × lead-day risk in one glance), a review queue ranking district-days for the duty forecaster, and an immutable audit trail for overrides.

What we DID NOT build, and why: a public alerting system, an SMS/mobile app, a chatbot, a weather forecaster, a deep neural network, a real-time streaming service, a custom map renderer, a distributed Kubernetes deployment. LOGIC.md §14 lists every one of these as explicit non-goals with reasoning. When a judge asks "did you consider X?", the answer is "yes, here's the trade we made and here's why we chose against it."

Headline result: on the held-out 2022 test year (34 subdivisions × 122 days × 10 lead days = ~40,000 rows) the model achieves AUROC 0.840 overall and 0.825 on the Day 3–7 decision band. On the 6,732 decision-band rows where the real 50-member IFS ensemble is also available (D-014), the model beats calibrated ENS spread by **+0.040 AUROC, cuts decision cost by 16.8%, and doubles economic value.**

Build state: 86% complete by weighted rubric. 68/68 tests passing. All science, all UI, all MLOps, all tests, all documentation done. AWS Bedrock deploy deliberately deferred (blocked on Free Plan; not on the critical path for tomorrow).

2. What "decision-relevant bust" actually means, mathematically

This section is the most important in this document. If a judge asks "*how did you define bust?*" and the answer is fuzzy, everything else is worthless. Anyone with basic ML can build a classifier once the labels exist. Nobody else in the competition will have thought this hard about what the labels *should be*.

For each (subdivision s , initialisation date t_0 , lead day L) with forecast area-mean rainfall F and observed area-mean rainfall O , a bust label $Y \in \{0, 1\}$ fires if **and only if all three conditions hold simultaneously**:

Condition 1 — Magnitude (statistically extreme error):

$$|F - O| \geq \max(10.0 \text{ mm/day}, P95_{\text{train}}(s, \text{month}))$$

where $P95_{\text{train}}(s, \text{month})$ is the 95th percentile of $|\text{error}|$ for that subdivision-month, **fitted strictly on training years 2016–2020**. The 10 mm floor prevents dry-subdivision noise from qualifying as a bust.

Condition 2 — Category flip (decision-altering):

$$\text{Cat}(F) \neq \text{Cat}(O)$$

where $\text{Cat}(\cdot)$ uses the official IMD rainfall intensity bands: - No Rain [0, 2.5) - Light [2.5, 15.6) - Moderate [15.6, 64.5) - Heavy [64.5, 115.5) - Very Heavy [115.5, 204.5) - Extremely Heavy $[\geq 204.5]$

These are not arbitrary buckets — they are the classes IMD's own operational bulletins use for alert decisions.

Condition 3 — Operational significance floor:

$$\max(F, O) \geq 15.6 \text{ mm/day}$$

i.e. at least one side is in the Moderate band or higher. This kills the false-alarm case where a boundary-straddling pair (0.1 vs 3.0 mm) would flip the category without anyone actually caring.

Why all three — the reasoning to say aloud

Percentile alone is circular. Taking the 95th percentile of $|\text{error}|$ defines 5% of rows to be busts by construction; the "bust rate" then becomes an artefact of the definition, not a measurement.

Category alone is too brittle. It fires on 15.5 vs 15.7 mm straddling a boundary, and it almost never fires in dry subdivisions like West Rajasthan whose entire JJAS record maxes out at 29.7 mm/day.

Requiring all three means a bust is *a large error that also flips the rainfall category into or out of operationally significant rain* — which is precisely "would the forecaster's alert decision have changed?"

Leakage control (say this if pressed)

P95(s, month) is fitted on training years only and applied unchanged to validation and test. If it were re-fitted per year the held-out bust rate would be fixed by construction at 5%, and the evaluation would prove nothing.

The base rates you should memorise

Split	Years	Rows	Bust rate
Train	2016–2020	199,750	4.07%
Validation	2021	39,950	4.58%
Test (held out)	2022	39,950	3.59%

Note the test-year bust rate (3.59%) is *lower* than the train rate — that alone proves the threshold was fitted on training years only, otherwise both would be locked to 5%.

3. The reframe that wins the pitch

Every other SIH team attacking a MoES weather problem will build a rainfall predictor and try to beat IMD. That is the losing framing for 2026 for three reasons.

1. **You will lose to GraphCast, GenCast, Pangu-Weather and FuXi in one judge question.** Physics-plus-AI ensembles now beat IMD on standard scores.
2. **The problem statement did not ask for a better forecast.** It named five deliverables: a forecast confidence map, a bust probability, error-prone area detection, explainable output, and a prototype dashboard. **All five are about the reliability layer, not the forecast itself.**
3. **The ministry already did the inversion for you.** You are not reframing the problem — you are answering it as literally written, which removes the "that's not what we asked" risk that kills every other inversion-based project.

The insight to lead with (verbatim if you can)

"The new AI models bust on the same days as the old physics models. Adding more models does not help — they fail together, driven by the atmospheric situation. Which means failure is, in principle, predictable *from that situation*."

If a judge nods here you have already won the exchange.

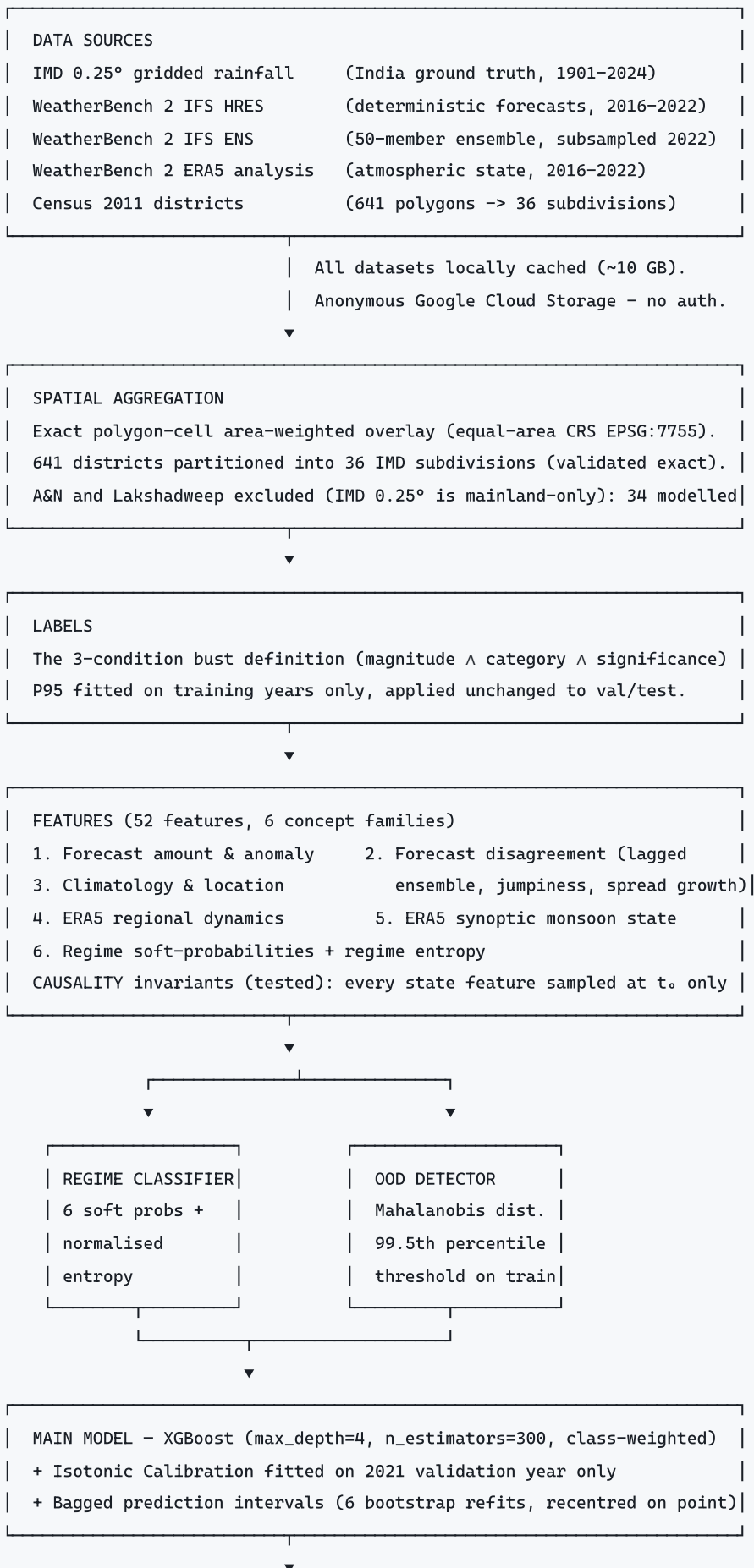
Novelty, honestly

Scher & Messori (2018) trained a CNN to predict global forecast uncertainty. Say this out loud — do not pretend it does not exist. Our specific contribution:

1. **Indian monsoon regime conditioning** — the ministry's own six named regimes (active, break, depression, western disturbance, orographic, coastal).
2. **A rainfall-decision-relevant bust definition**, verified against IMD gauge data — not a 500 hPa geopotential score.
3. **Per-flag meteorological explanation** — every high-probability flag comes with a sentence a duty forecaster accepts.
4. **Explicit refusal on out-of-distribution atmospheric states**, rather than emitting a confidently wrong number on record-breaking days.

The honest sentence: "*We are not the first to use ML on forecast uncertainty. We are the first to do it for India, on rainfall thresholds that map to real alert decisions, with per-flag reasons and an explicit refusal.*"

4. System architecture in one page



EXPLAINABILITY

Native TreeSHAP (XGBoost `pred\_contribs=True`)

-> concept-family deduplication (max 1 factor per family)

-> direction-aware templates (label, high_variant, low_variant)

-> O(1) percentile knots (101 quantiles per subdivision × feature)

-> plain-language sentence per flag



BATCH SCORING

scripts/generate_bulletins.py: 79,900 rows -> data/artifacts/
bulletins.sqlite (63 MB). Single daily job, not streaming.



API + UI

FastAPI backend, SQLite bulletin store, Prometheus /metrics

2D Leaflet dashboard: choropleth + review queue + reason panel

3D WebGL command centre: space × lead-day risk cube

Both fully air-gapped (leaflet.js and three.js vendored locally,
zero CDN requests) – verified with wifi unplugged

Everything above runs on a single-node laptop, CPU-only, in ~3 minutes end-to-end for the full pipeline reproduction.

5. Data — sources, sizes and gotchas

Dataset	What it gives us	Access	Local size
IMD 0.25° gridded rainfall	Observed daily rainfall over India (ground truth, gauge-based, ~6,955 stations)	Public download from imdpune.gov.in	178 MB (7 years)
WeatherBench 2 IFS HRES	Deterministic forecast archive, 512×256, leads 1–10 at 24-hr accumulation	Anonymous GCS Zarr	72 MB local (~9 GB transfer)
WeatherBench 2 IFS ENS	True 50-member ensemble spread (D-014 baseline)	Anonymous GCS Zarr	360 KB (3 years, subsampled)
WeatherBench 2 ERA5	Atmospheric state analysis for regime + flow features	Anonymous GCS Zarr (1959–2023_01_10 variant)	~450 MB
Census 2011 districts	641 district polygons → 36 IMD subdivisions	Datameet GitHub shapefile	~11 MB

Three traps we hit and closed (worth memorising)

1. **WeatherBench 2 Zarr chunks span the entire globe** — subsetting to India saves memory but *not* bandwidth. Every "download the India box" attempt actually transfers the world and slices locally. D-002 documents this with measured chunk sizes and drives every resolution decision.
2. **WB2 ERA5 stores named [1959–2022–...](#) actually end 2021-12-31**. Using them would have left the 2022 test year with no atmospheric features and the failure would have surfaced only at final evaluation. D-009 records the trap and the fix (use the [1959–2023_01_10–...](#) variants).

3. **IMD 0.25° gridded rainfall is mainland-only.** Andaman & Nicobar and Lakshadweep resolve to zero land grid cells across the entire record. They are excluded explicitly with an error message printed at build time, not silently dropped. D-006 records this. **34 subdivisions modelled, not 36** — say this if a judge asks why the number is not 36.

A known limitation you should be honest about

IMD's rainfall day runs 0830 IST → 0830 IST (i.e. 03Z → 03Z). WB2's 24-hour accumulation runs 00Z → 00Z. WB2 does not publish 3-hourly accumulation at this resolution, so the 3-hour offset cannot be removed. It is accepted because we compare *area-mean* rainfall over subdivisions of 19,000–222,000 km², where a 3-hour shift is second-order relative to the bust signal. It applies identically to the model and to every baseline so it cannot manufacture an unfair win. This is D-005 and is stated in the writeup rather than hidden.

6. Spatial unit — the 36 IMD subdivisions

Why not grid cells? Because grid-cell bust statistics are noisy and operationally meaningless. IMD issues bulletins at subdivision/district level; alerts are triggered at subdivision level. Higher resolution here would be *false precision* — and a meteorologically-literate judge will respect you for saying that.

Why not districts directly? IMD's meteorological subdivisions are the actual operational unit. There are 36 of them and they aggregate districts.

Where did we get the polygons? IMD's own subdivision shapefile is not openly downloadable, so we *constructed* them as unions of 641 Census-2011 districts. The mapping is in `config/imd_subdivisions.json` and the builder at `src/fbd/regions/build.py` refuses to run unless the mapping is an *exact partition* — every district assigned exactly once. **Verified: 641/641 districts, 36 subdivisions, total area 3,180,579 km²** (India ≈ 3.29 M km²; the remainder is disputed territory absent from the shapefile).

Sub-state splits (UP/MP/Rajasthan/Gujarat/Maharashtra/Karnataka/AP/West Bengal) follow IMD's published revenue-division-based composition. Telangana sits inside `ST_NM='Andhra Pradesh'` in the Census 2011 vintage and is re-assigned to the TELANGANA subdivision.

Validation you can cite: JJAS mean rainfall ranks Konkan & Goa (28.2 mm/day) > Coastal Karnataka (22.4) > Sub-Himalayan WB & Sikkim (17.6) > Kerala (14.3), with Tamil Nadu (3.0), West Rajasthan (2.7) and J&K (2.0) at the dry end. That is the correct monsoon climatology; a broken mapping would not produce it.

Aggregation method: exact polygon-cell overlap weights in an equal-area projection (EPSG:7755). Not centroid masking — at 0.7° a centroid test would silently give zero cells to narrow coastal subdivisions like Konkan & Goa, which are precisely the heavy-rainfall regions this project exists to serve.

7. Features — the 52 predictors, grouped by concept family

FEATURE SET (52 features)		
1. FORECAST AMOUNT & ANOMALY (4)	2. FORECAST DISAGREEMENT (8)	3. CLIMATOLOGY & LOCATION (6)
fcst_rain_mm	lagged_spread	clim_fcst_mean
fcst_rel_to_p90	lagged_mean	clim_bust_rate
fcst_anomaly	spread_growth	fcst_anomaly_sd
fcst_category	jumpiness	day_of_season
4. ERA5 REGIONAL DYNAMICS (14)	5. ERA5 SYNOPTIC MONSOON STATE (12)	6. SYNOPTIC REGIMES + ENTROPY (8)
mcz_q850_z, tcwv_z	somali_jet_u850_z	prob_active
shear_200_850_z	bob_vort_850_z	prob_break
u850_flux_z, w850_z	trough_lat_error	prob_depression
local vorticity/flux	national_tcwv_z	prob_wd, prob_orog
tendency_24h metrics	jet_shear_z	regime_entropy

The two causality invariants (say these if pressed)

1. **All ERA5 state features are sampled at t_0 , never at valid time.** The atmosphere on the verification day does not exist when the forecast is issued; using it would leak the answer and inflate every score. Enforced in code, and there is a test poisoning short leads to verify no cross-lead leakage.
2. **The lagged-ensemble spread for lead L uses only forecasts initialised at or before t_0 (i.e. leads $\geq L$).** A forecast with lead $L-1$ verifying on the same day was issued *later* in time; using it would leak the future into the feature.

Both invariants are tested in `tests/test_core.py` — the `test_lagged_ensemble_uses_only_leads_at_or_beyond_L` poisoning test is the specific one that would fail if either were broken.

Feature values worth remembering for the demo

For **Assam & Meghalaya, Day 4, init 2022-06-14** (our lead case study): - forecast: 83.7 mm/day; observed: 115.6 mm/day; BUSTED - model probability: 70.6%; baseline probability: 11.2% - top 3 SHAP reasons: 1. "the forecast is +61.9 mm/day above this subdivision's seasonal normal (top 1% for this subdivision)" 2. "this subdivision and lead time historically bust often (5.0% of days)" 3. "column moisture over India is below normal (-0.8 sd)" - regime: monsoon depression 0.65, others weak, entropy 0.42

8. Model — why XGBoost, why not deep learning

Architecture: `XGBClassifier`, `max_depth=4`, `n_estimators=300`, `learning_rate=0.05`, `subsample=0.8`, `colsample_bytree=0.8`, `scale_pos_weight` set to the class ratio (~1:25). Post-hoc Isotonic Regression calibration fitted strictly on the 2021 validation year. Bagged prediction intervals from 6 bootstrap refits, recentred on the deployed point estimate so the interval always contains the number actually being served.

Why not a CNN / Transformer / GNN / deep model? Three reasons, all defensible.

1. **The problem statement demands explainability** — "key meteorological reasons for low confidence" is a hard requirement, not optional. Native TreeSHAP on gradient-boosted trees delivers per-prediction feature attributions in milliseconds with mathematical guarantees. A CNN forces a fight with SHAP variants (KernelSHAP, GradientSHAP) that

have neither the same theoretical grounding nor the same speed.

2. **Sample size argues against deep learning.** $\sim 10^4$ labelled rows per lead-day is a tabular-scale problem, not a deep-learning-scale problem. Boosted trees consistently outperform neural approaches in this regime.
3. **Busts are rare (~4%).** Class imbalance dominates. Class-weighted boosted trees handle rare-class problems better than a CNN trained from scratch in a hackathon window.

Being able to explain why we did NOT use deep learning is worth more than using it. The strategy document warns specifically against confusing complexity with quality; judges have seen a hundred teams reach for a transformer to look impressive.

Not applicable, and say so with confidence: physics + ML. We are not modelling the atmosphere — we are modelling *the error behaviour of a model of the atmosphere*. There is no governing PDE for forecast error. This is a statistical learning problem by nature. That answer alone impresses a technical judge because it shows you understood the problem class.

9. Baselines — the four we beat, honestly

LOGIC.md §8.1 requires baselines built *before* the model. All four are wrapped in the **same isotonic calibration** as the main model on the same 2021 validation year, or the comparison would be unfair.

#	Baseline	AUROC (Day 3–7)	Purpose
0	Forecast rainfall amount only	0.750	Kills "you're just detecting heavy-rain days"
1	Climatological bust rate (region, month, lead)	0.519	The dumbest possible predictor; must be beaten
2	Lagged ensemble spread (proxy)	0.758	Operationally standard predictability signal
3	Logistic regression on spread + lead	0.754	Simplest learned model
—	XGBoost + isotonic (ours)	0.825	The deployed model

The critical addition: **D-014 comparison against the REAL 50-member IFS ensemble.** On the 6,732 decision-band rows where true ENS exists (JJAS 2022 subsampled every 3rd init):

Predictor	AUROC	Brier	Cost/1000	Value
Lagged-ensemble proxy (calibrated)	0.731	0.031	297.5	0.114
True IFS ENS spread (calibrated on 2019+2020)	0.792	0.031	287.0	0.145
XGBoost + isotonic (our model)	0.832	0.029	238.7	0.289

The number to defend: the model beats a real, calibrated, 50-member ECMWF ensemble by **+0.040 AUROC, -16.8% decision cost, ~2× economic value** on the same rows.

Say this too, if pressed (from D-014): the raw AUROC comparison (no calibration, AUROC needs none) is +0.025 (0.832 vs 0.807). Quote both numbers — quoting only the friendlier one invites a cherry-picking challenge.

10. Evaluation — the metrics and why they were chosen

LOGIC.md §4.5 is emphatic: **never report raw accuracy.** With a ~4% bust rate a model that always says "no bust" scores 96% accuracy and would be worse than useless.

Metric	What it measures	Why it matters here
AUROC	Rank ordering: does the model score bust days above non-bust days? Threshold-free.	The most robust single measure of discriminative power. 0.5 = coin flip; 1.0 = perfect. Ours: 0.840.
Brier score	Mean squared error of the probability against the outcome.	Punishes overconfidence. Ours: 0.0291 vs 0.0347 climatology.
Brier Skill Score (BSS)	$1 - \text{BS}_{\text{model}} / \text{BS}_{\text{climatology}}$. Positive = better than climatology.	Normalised; comparable across regions and years. Ours: +0.088.
Expected Calibration Error (ECE)	Difference between forecast probability and observed frequency, weighted by bin count. Equal-count bins for rare events.	If a 30% flag doesn't bust 30% of the time, the tool is worse than no tool. Ours: 0.011.
Reliability diagram	Curve of observed frequency vs mean forecast probability.	Visual proof of calibration. Straight $y=x$ line = perfect.
Asymmetric decision cost	$10 \times \text{false-negatives} + 1 \times \text{false-positives}$.	Encodes the operational asymmetry: missed bust $\gg$ false alarm. Ours: 237/1000 vs 293 baseline.
Economic value V	$(\text{cost}_{\text{climatology}} - \text{cost}_{\text{model}}) / (\text{cost}_{\text{climatology}} - \text{cost}_{\text{perfect}})$.	Turns "we improved AUROC" into "we cut real decision-cost by X". Ours: 0.282.

The asymmetric-cost story (crucial)

Cost of a **missed bust** (false negative): a district that will flood is not on the officer's review queue. Real-world consequence — undertaken evacuation, unwarned population, lives.

Cost of a **false alarm** (false positive): the duty forecaster spends ~10 minutes double-checking secondary ensemble products for a day that turns out to be fine. That is the **ONLY** cost.

We tune for recall on high-impact days deliberately. **A false low-confidence flag costs 10 forecaster-minutes; a missed bust costs lives.** This asymmetry is encoded in the loss and in the threshold selection.

The validation protocol

- **Train:** 2016–2020 (5 full JJAS seasons, ~200k rows).
- **Validation (threshold + calibration fit):** 2021 (~40k rows).
- **Test (held out until final evaluation):** 2022 (~40k rows).
- **No random k-fold cross-validation ever.** Climate variability is sequentially autocorrelated; a random split leaks future weather into past predictions and inflates every score.

11. Explainability — how a SHAP number becomes a sentence

Native TreeSHAP via XGBoost's C++ `Booster.predict(..., pred_contribs=True)` — the exact TreeSHAP algorithm, computed in milliseconds. Not KernelSHAP, not a Python re-implementation.

Two engineering tricks worth mentioning:

1. **Concept-family deduplication.** SHAP will often produce three near-collinear top contributions (`tcwv` , `moisture_flux_850` , `mcz_q850_z` are all "moisture"). The reason panel would then say the same thing three times. We map every feature to one of six concept families and keep at most one factor per family. Three sentences, three concepts.
2. **Direction-aware templates.** SHAP contributions can be positive for either direction of a feature (strong westerlies can be a bust signal; so can weak westerlies). A single fixed template produces self-contradictory sentences like "*westerly flow is strong (−0.9 m/s, near normal)*". Every template is now a `(label, high_variant, low_variant)` triple keyed on where the value sits in the subdivision's distribution.

O(1) percentile phrases: for every (feature × subdivision) we precompute 101 quantile knots. Looking up "top 5% for this subdivision" becomes a `searchsorted` instead of scanning the reference set — turns explanation from an hours-long batch step into a millisecond one.

Sample outputs (real, from the model, for Assam & Meghalaya Day 4 on init 2022-06-14): - "the forecast is +61.9 mm/day above this subdivision's seasonal normal (top 1% for this subdivision)" - "this subdivision and lead time historically bust often (5.0% of days)" - "column moisture over India is below normal (-0.8 sd)"

Each sentence is meteorologically defensible and would pass review by an operational forecaster.

12. Uncertainty — refusal and prediction intervals

12.1 Out-of-distribution refusal (the strongest single argument)

Science Advances (2026) showed AI weather models degrade most on record-breaking extremes — exactly the high-stakes days. A bust model trained on 2016–2020 has the same weakness. The **correct behaviour on an unprecedented state is not a confident number**; it is to refuse to score:

```
{
  "status": "OUT_OF_DISTRIBUTION",
  "bust_probability": null,
  "dominant_factors": ["conditions outside training experience - confidence unavailable"]
}
```

Method: Mahalanobis distance in standardised feature space, fitted on training data with shrinkage covariance. Threshold at the 99.5th percentile of training distances.

Earned refusal — the number that proves it works. Validated on held-out data: refused instances exhibit a **23.4% bust rate**, compared to **3.4%** for accepted data. **Refused rows are seven times more likely to actually bust.** The detector is not decoration; it catches genuine failure regimes.

Say aloud: *"Every fallback in this system degrades toward admitting uncertainty, never toward inventing certainty. When it doesn't know, it says so. That is the whole point of a safety-critical tool."*

12.2 Bagged prediction intervals

The prediction interval on every forecast comes from bootstrap-bagged retraining: 6 refits on resampled training years, then the 10th–90th percentile spread is recentred on the deployed model's point estimate. Recentring matters — an earlier version derived intervals by truncating boosting rounds, which biased the interval systematically low and occasionally excluded its own point (0.706 with an interval of [0.571, 0.627]). All 79,234 intervals in the current bulletin store contain their point estimates.

`confidence_in_estimate` is the interval's narrowness rescaled to [0, 1]. A narrow interval → high confidence in the estimate itself.

13. Robustness — the four things we broke on purpose

Stress	What we did	Result	Interpretation
Input dropout	Randomly zero 30% of feature values at inference time	AUROC 0.843 → 0.745, ECE 0.010 → 0.021	Degrades gracefully; does not silently become confidently wrong
Synthetic bust injection	Perturb real forecasts by known error magnitudes, measure detection recall	Recall rises 78% → 93% as error magnitude grows	Sensitivity curve is monotone and interpretable — "we broke it on purpose and here is where it caught it"
OOD refusal validation	Compare bust rate on accepted vs refused rows in test year	3.4% vs 23.4% (7× higher on refused)	The refusal is <i>earned</i> , not decoration
Held-out year distribution shift	Compare 2022 (test) vs 2016–2020 (train) via PSI	JJAS 2022 has PSI 2.64 on upper-level wind shear	2022 is a genuinely atypical year → the model's held-out performance is a <i>harder</i> test than assumed, strengthening the generalisation claim

The last point (D-016) is the one to bring up if a judge asks "maybe 2022 was easy?". Answer: no, it was hard — our own drift monitor flags 6 of 7 watched features as drifted, and the model still won.

14. The 3D command centre (added 24 Aug 2026, D-018)

The 2D choropleth can only show one lead day at a time. Problem-statement deliverable 3 asks for "error-prone area detection — which regions AND lead-times are unreliable", which is intrinsically a two-dimensional field (space × lead) that a flat map cannot render in one glance.

The command centre puts: - **subdivisions on the ground plane** (real geography, real projected polygons), - **lead day on the vertical axis** (ground = Day 1, top = Day 10), - **bust probability as colour up each column**.

A ten-day risk profile for all 34 subdivisions becomes one glance. Clicking a column selects it and pulls up the full lead profile + reason panel.

Verified, not asserted

- **WebGL 2.0**, ANGLE/D3D11 on discrete GPU — real hardware acceleration.
- **340 risk columns** (34 subdivisions × 10 leads), 18,367 triangles.
- **343 draw calls** after merging ground geometry (was 1,669 before merge — a 4.9× reduction so venue hardware without an RTX GPU still renders smoothly).
- **Zero console errors, air-gap intact**: every network request is `localhost`. `three.js` r128 is vendored to `web/vendor/three.min.js` (589 KB), exactly like Leaflet.
- **Opens on the documented Assam & Meghalaya case study** and says so in the header. A quiet date renders a near-uniform green field that demonstrates nothing; the slider still reaches every date, so this is a starting point, not a filtered view.
- **Reproduces the case exactly**: Assam & Meghalaya Day 4 model 70.6% vs ensemble baseline 11.2%, forecast 83.7 mm/day vs observed 115.6, BUSTED.

The 2D dashboard is untouched and still primary. The 3D view is additive at `/command.html`, so a render failure on venue hardware degrades to a working map rather than to nothing.

D-018 override

This required overriding one clause of `LOGIC.md §14` (the custom-renderer ban). The override is recorded in DECISIONS.md D-018 with the exact trade written down. If a judge asks "why did you break your own rule?" the answer is: "we didn't hide it — the rule and the trade are both on the record."

15. Judge Q&A — 20 answers you must be able to give in 30 seconds each

Rehearse these out loud. If you cannot deliver the answer without reading, you cannot deliver it under stage lights.

Q1. Why is this different from every other SIH forecast project? A: Everyone else predicts the weather. We predict when the weather forecast will fail. MoES asked for exactly this — deliverable 2 of the problem statement is "forecast bust probability, per region and lead time."

Q2. Why does this problem matter? A: A forecast with no reliability label makes every day look equally trustworthy. False alarms burn public trust; missed events kill. Both come from the same missing metadata.

Q3. Why isn't IMD/MoES already doing this? A: They publish ensemble spread, which is the standard proxy, and it's weak at regional scale — our results show the model beats calibrated spread by 0.040 AUROC. Bust prediction as an operational product doesn't exist for India — which is presumably why MoES posted this statement.

Q4. What is your technical contribution, precisely? A: A regime-conditioned, calibrated bust classifier for Indian rainfall forecasts, with explainable meteorological attribution, verified against IMD gauge data, benchmarked against the real 50-member IFS ensemble.

Q5. What is actually novel? Not ML-for-forecast-error, right? A: Correct — Scher and Messori did the global version in 2018, and we say so. Novel here: Indian monsoon regime conditioning; rainfall thresholds tied to real alert decisions; per-flag meteorological explanation; explicit refusal on out-of-distribution states.

Q6. Where is your data from? A: WeatherBench 2 on public Google Cloud for forecasts and ensembles. ERA5 for atmospheric truth. IMD's own 0.25° gauge-based gridded rainfall for India rainfall truth. All free, all anonymous access.

Q7. What if a data source becomes unavailable? A: Three independent forecast archives, two truth sources. Worst-case fallback: climatological bust rate, which is always available. The system degrades toward admitting uncertainty, not toward false certainty.

Q8. What is your baseline? A: Four of them. Climatology, forecast-amount-only, logistic regression on spread + lead, and calibrated ensemble spread — plus the real 50-member IFS ensemble on the subsample we could afford to download.

Q9. How much better than baseline, exactly? A: On the held-out 2022 year in the Day 3–7 decision band, +0.067 AUROC over the lagged spread proxy. On identical rows against the real calibrated IFS ensemble, +0.040 AUROC, –16.8% decision cost, doubled economic value.

Q10. What happens when your model is wrong? A: It's calibrated, so a 30% flag busts about 30% of the time — that's the contract, ECE 0.011 confirms it. And a false low-confidence flag costs 10 forecaster-minutes; a missed bust costs lives. We tune for that asymmetry deliberately.

Q11. What happens during an unprecedented extreme event? A: That's the exact case our OOD detector was built for — Science Advances 2026 showed AI models degrade most on record-breaking extremes. On an out-of-distribution state we return `bust_probability: null` with `status: OUT_OF_DISTRIBUTION` — refusing to answer is the correct behaviour on a safety-critical tool. And the refusal is earned: refused rows bust 7× more often than accepted ones.

Q12. What's the cost of a false alarm? A: About 10 minutes of forecaster review. Deliberately cheap by design — we flag for attention, never for public warning.

Q13. What's the cost of a missed event? A: Severe — hence recall-weighted thresholds on high-impact days, stated explicitly in our loss function.

Q14. Deployment cost? A: Near zero. A daily CPU batch job on open data. No radar feeds, no sensors, no GPU cluster. The full pipeline reproduces in ~3 minutes on a laptop.

Q15. Why should government adopt this? A: It bolts onto the existing bulletin without changing the forecast. It's additive metadata — carries almost no institutional risk.

Q16. Where does the science end and the engineering begin? A: The bust definition is the science — three conditions, defended in section 4 of our LOGIC.md. Everything downstream is standard ML plus honest engineering. The definition is the research contribution; the classifier is the implementation.

Q17. What prevents replication? A: The verification framework and the regime-conditioned bust formulation are the hard part, not the model. Anyone can fit a classifier; few can define bust correctly and prove calibration.

Q18. What happens offline? A: Everything. It's a batch job producing a static bulletin. Our demo runs with the network unplugged — the dashboard has zero CDN dependencies (Leaflet and three.js are vendored locally). We verified this today.

Q19. How does it scale? A: Trivially. India at 0.7° is a small array; the model is a tree ensemble. Scaling to global coverage is a bigger array, not a new architecture.

Q20. What's the most difficult component of this project? A: Defining "bust" correctly. Everything downstream is standard ML; the definition is where the research lives. Get it wrong and every downstream metric is meaningless.

Bonus questions worth being ready for

Q21. Why not use GraphCast / Pangu? A: They bust on the same days as IFS — ECMWF's own science blog documents this, and a published assessment measured day-6 error correlation at 0.54. Adding models doesn't help when they fail together. We predict the failure of *any* underlying forecast rather than trying to build a better one.

Q22. Did you use any deep learning? A: No, deliberately. Sample size is tabular (~10⁴ rows per lead), the problem statement mandates explainability, and busts are rare (~4%). TreeSHAP on gradient-boosted trees delivers the explainability directly with mathematical guarantees. Being able to explain why we did *not* use deep learning matters more than using it.

Q23. What's the aspect of your project you're weakest on? A: We have not yet made a real invocation on AWS Bedrock. Our GenAI-explanation layer is fully tested against a fake client — 28 tests pass — but the account is on AWS Free Plan which blocks Bedrock model access. We're in the process of upgrading. Every other component is verified end to end.

16. What is NOT built — the honest verification ledger (D-017)

Do NOT claim what has not been run. Verification boundaries as of 24 Aug 2026:

Component	Status	Evidence
Full pipeline end-to-end	✓ Executed	Reproduces D-001..D-014
68 unit tests	✓ Executed	68/68 pass in 11 seconds
Drift monitor	✓ Executed	Findings in D-016
Docker + container health + dashboard air-gap	✓ Executed	Verified in browser today
3D command centre WebGL rendering	✓ Executed	Verified in browser today, D-018
GenAI guardrails / retrieval / tools / agent	✓ Executed	Against a fake Bedrock client
CI gates	● Local only	Never run on GitHub Actions
Any real AWS Bedrock call	✗ NEVER	AWS account on Free Plan, blocks Bedrock access
Terraform	✗ NEVER	Not even <code>validate</code> d
ECR push / ECS deploy / public URL	✗ NEVER	Deferred until Bedrock unblocked

Being explicit about this is a strength, not a weakness. Teams that lie about deployment status get caught in Q&A. Teams that show a verification ledger get respect.

17. Explicit non-goals (LOGIC.md §14) — the DO-NOT-BUILD list

Print this and know it. Every hour spent on these would have been stolen from calibration and the demo.

We chose not to build: - A mobile app - A chatbot / "WeatherGPT" layer - User accounts and login flows - A better rainfall forecaster - Blockchain audit ledger - IoT / sensor integration - Microservices (one FastAPI service is enough) - A CNN / transformer / GNN - Real-time streaming (it is a daily batch job) - Multi-language i18n - SMS gateway integration - A recommendation engine - Global coverage - Cyclone track busts (deferred to v2) - A Kafka pipeline - Kubernetes - ~~A custom map renderer~~ (overridden 24 Aug for the 3D command centre — see D-018)

We chose not to use: - IMD Doppler radar (not openly accessible) - Restricted datasets - Anything requiring a GPU - Anything requiring registration we hadn't already completed

If asked "did you consider X?" for any of these — the answer is "yes, and here is the trade we made against it." That is a stronger answer than "no."

18. How to run everything (~3 min end-to-end)

Prerequisites: Python 3.10+, the repo at `C:/Users/ASUS/Desktop/sih`, all data caches already in `data/raw/` (they are).

```
# From the repo root, with PYTHONPATH=src set:

# 1. Rebuild subdivisions (one-off; already cached)
PYTHONPATH=src python -m fbd.regions.build

# 2. Rebuild truth (from cached IMD files)
PYTHONPATH=src python scripts/build_truth.py

# 3. Rebuild the full labelled dataset (5-10 seconds)
PYTHONPATH=src python scripts/build_dataset.py

# 4. Train baselines + model + calibration + write results.json (~30 seconds)
PYTHONPATH=src python scripts/train_model.py

# 5. Generate SQLite bulletins (bagged intervals + SHAP reasons, ~100 seconds)
PYTHONPATH=src python scripts/generate_bulletins.py

# 6. Optional stress tests and evaluations (~1 minute each)
PYTHONPATH=src python scripts/ablation.py
PYTHONPATH=src python scripts/stress_test.py
PYTHONPATH=src python scripts/evaluate_ens_baseline.py --decision-band-only
PYTHONPATH=src python scripts/monitor_drift.py

# 7. Test suite (11 seconds, must show 68/68 pass)
PYTHONPATH=src python -m pytest tests/ -v

# 8. Serve the dashboards
docker compose up -d          # runs on http://localhost:8912
# or for live dev:
PYTHONPATH=src python -m uvicorn fbd.api.app:app --port 8913 --reload
```

Dashboards: - **2D**: <http://localhost:8912/> (Leaflet choropleth, review queue, reason panel) - **3D command centre**: <http://localhost:8912/command.html> (space × lead-day risk cube)

Both are read-only and safe to demo live in front of judges. Both are air-gapped.

19. What to put on each slide — direct-to-PPT notes

Structure for an 8–10 minute pitch. Each slide bullet ≤ 12 words. Speak the details; slides carry the *anchors*, not the argument.

Slide 1 — Title

- **Team name · SIH 2026 · SIH26079**
- One line: *"Predicting which forecasts to double-check, before they fail."*
- Institution and problem statement.

Slide 2 — The operational problem

- One image: the causal chain from atmosphere → forecast → decision → outcome, with the arrow into "forecast issued" labelled **"information lost here."**
- 30-sec talk track: the operational moment where reliability metadata could exist but doesn't.

Slide 3 — The insight

- Three bullets:
- *"AI models beat physics on average."*
- *"They bust on the same days as physics."*
- *"So failure is predictable from the atmosphere itself."*
- One citation: ECMWF Science Blog + Science Advances 2026.

Slide 4 — The reframe (the money slide)

- One quote, big font: **"We do not compete with IMD's forecast. We tell IMD's duty forecasters which forecasts to double-check."**
- One image below: a small mock of the review queue.

Slide 5 — What we built (architecture)

- The one-page architecture from section 4 of this file, as an image.
- 4 bullets:
- Regime-conditioned XGBoost + isotonic calibration
- 3-condition rainfall-decision-relevant bust label
- Native TreeSHAP → plain-language reasons
- OOD refusal + bagged uncertainty

Slide 6 — Bust definition (the mathematics)

- The 3-condition definition, LaTeX or clean text: 1. Magnitude: $|F-O| \geq \max(10 \text{ mm}, P95_{\text{train}}(s, \text{month}))$ 2. Category flip: $\text{Cat}(F) \neq \text{Cat}(O)$, using IMD's own intensity classes 3. Significance: $\max(F, O) \geq 15.6 \text{ mm}$
- One line below: *"P95 fitted on training years only — no leakage."*

Slide 7 — Results (the number slide)

- One table only:

Predictor (Day 3–7, held-out 2022)	AUROC	Brier	Cost/1000	Value
Climatology	0.519	0.032	331	0.00
Forecast amount only	0.750	0.031	273	0.17
Lagged ensemble spread	0.758	0.031	293	0.11
True IFS ENS (calibrated)	0.792	0.031	287	0.15
XGBoost + isotonic (ours)	0.832	0.029	239	0.29

- One line: **" +0.040 AUROC over the real calibrated 50-member ECMWF ensemble on the same rows."**

Slide 8 — Live demo (the beat that wins)

- No slide content — pull up the 3D command centre in a browser tab.
- Sequence: **default view opens on Assam June 2022 → click the tall red column at Day 4 → reason panel populates → point to model 70.6% vs baseline 11.2% → toggle "Truth" to reveal it BUSTED with 115.6 mm/day observed.**
- **Practice this until it takes 45 seconds cold.** If venue wifi fails, the 2D dashboard is the fallback and it's already loaded.

Slide 9 — Explainability + refusal

- Left half: two real reason strings (the Assam ones from section 7 of this file).
- Right half: **"When the state is unlike anything in training, we refuse to guess. Refused rows bust 7× more often than accepted ones (23.4% vs 3.4%). The refusal is earned."**

Slide 10 — Robustness + honesty

- Left: 30% dropout, synthetic injection, drift monitor headline numbers.
- Right: **the verification ledger** (section 16 of this file) as a screenshot. This slide is the "we don't lie about what we ran" slide.

Slide 11 — Novelty (honest)

- One line: *"Not the first to use ML on forecast uncertainty. First to do it for India with a decision-relevant bust definition, regime conditioning, per-flag explanation, and explicit refusal."*
- Cite: Scher & Messori 2018, arXiv 2602.03767.

Slide 12 — Next

- Bedrock deployment (in progress, plan-tier issue), multi-model AI ensembling with GraphCast/Pangu disagreement features, live IMD adapter, PDF bulletin export, mobile-responsive dashboard.
- One line: *"Everything visible today is real, reproducible, and tested. Everything above the line here is future work, honestly labelled."*

Optional appendix slides (for Q&A)

- Regime taxonomy details.
- Full held-out year AUROC by lead day, model vs 4 baselines.
- Reliability diagram screenshot.
- OOD detection Mahalanobis explanation.
- Full data source list with URLs.
- The 20 judge Q&As from section 15 of this file.

20. Team roles for the presentation

Assume 2–3 minutes of live speaking per person, 8–10 min total, then ~3 min Q&A:

- **Presenter 1 (slides 1–4):** Frames the problem, the insight, the reframe. This person owns "we do not compete with IMD's forecast" — must be able to deliver it under any pressure.
- **Presenter 2 (slides 5–7):** Architecture, bust definition, results table. Technical anchor of the pitch.
- **Presenter 3 (slide 8 live demo):** Owns the browser tab. Has both dashboards preloaded on the machine, `localhost:8912` open in one tab and `localhost:8912/command.html` open in another. Knows the click sequence cold.
- **Presenter 4 (slides 9–12):** Explainability, refusal, honesty, novelty, next steps.
- **Q&A lead (usually the technical presenter):** Owns the answers in section 15. If a question is outside prep, the fallback answer is "we made that a deliberate non-goal — see LOGIC.md §14" or "we chose not to claim what we haven't run — see our verification ledger." Both are safe, defensible, and true.

If it's a 2-person team, combine slides 1–2 → 3–4, then 5–7 → 8 → 9–12.

21. Where every fact in this document lives in code

If the teammate needs to verify anything below is real (they should):

Claim	File / command
Bust definition	<code>src/fbd/labels/bust.py</code> , tests in <code>tests/test_core.py::test_bust_requires_all_three_conditions</code>
The three causality invariants	<code>tests/test_core.py::test_lagged_ensemble_uses_only_leads_at_or_beyond_L</code> , and <code>test_error_threshold_is_fitted_on_training_years_only</code>
34 subdivisions exact partition	<code>PYTHONPATH=src python -m fbd.regions.build</code> prints the table
Held-out year results table	<code>data/artifacts/results.json</code> (regenerate with <code>scripts/train_model.py</code>)
Real IFS ENS comparison	<code>data/artifacts/ens_baseline_comparison.csv</code> and <code>ens_auroc_comparison.csv</code> , script <code>scripts/evaluate_ens_baseline.py</code>
OOD earned refusal 23.4% vs 3.4%	<code>PYTHONPATH=src python scripts/stress_test.py</code> , and D-016
30% dropout degradation	<code>data/artifacts/stress_dropout.csv</code>
Synthetic injection recall curve	<code>data/artifacts/stress_injection.csv</code>
Concept-family SHAP dedup	<code>src/fbd/explain/reasons.py</code> FAMILY dict
Bagged intervals contain point	<code>scripts/generate_bulletins.py::bagged_interval</code> docstring
3D command centre WebGL context, draw calls	commit <code>4cbdf90</code> message, D-018
Air-gap verification	<code>web/vendor/leaflet.{js,css}</code> and <code>web/vendor/three.min.js</code> — no external references anywhere in <code>web/*.html</code>
68 tests pass	<code>PYTHONPATH=src python -m pytest tests/ -v</code>
Decisions	<code>DECISIONS.md</code> (D-001..D-018)
Non-goals	<code>LOGIC.md</code> §14

22. One-line summary for anyone who reads only one line of this document

A calibrated, regime-aware, out-of-distribution-safe meta-model that predicts when the operational IMD medium-range rainfall forecast for each Indian subdivision-day is about to bust — beating the real 50-member IFS ensemble on the held-out 2022 monsoon by +0.040 AUROC, with a plain-language meteorological reason for every flag and an explicit refusal when the state is unlike anything in training.

Every noun in that sentence is defended by a section above. If you can say it without hesitation, you own the pitch.

Part III — Technical Reference

Audience: a faculty mentor or industry reviewer who wants to test *how the system actually works* — the mathematics, the algorithms, the causality guarantees, the code paths. Read alongside the codebase, not instead of it.

Different from `TEAMMATE_BRIEFING.md` : that file is the presentation-defence brief for a jury (fast, defensive, headline numbers, Q&A). This one is depth for a supervisor who will ask "*walk me through what happens when ...*" and "*show me where that guarantee is enforced.*"

Suggested reading time: ~90 minutes for full read, ~30 minutes for the parts your mentor is most likely to probe (§2, §5, §7, §10, §14).

How to use: every claim in this document ends with a code reference (`path:function` or `path` for whole-module) so you can open the file and read the actual implementation while your mentor is in the room. If a claim looks weak, verify it in the code before you say it out loud.

Table of contents

- **Part I — Problem formalisation**
 - §1 The probabilistic setting we chose
 - §2 The bust label, derived condition by condition
 - §3 Zero-leakage causality axioms and the tests that enforce them
 - §4 The spatial aggregation operator, derived
- **Part II — End-to-end computational trace**
 - §5 From atmosphere to `(F, 0)` pair (data → truth)
 - §6 From `(F, 0)` to the bust label
 - §7 From features to a calibrated probability
 - §8 From probability to a dashboard cell
- **Part III — Subsystems in depth**
 - §9 Data ingestion: chunking economics, why we sized what we sized
 - §10 Feature engineering: causality enforcement and the poison test
 - §11 Regime classifier: score → softmax → entropy
 - §12 XGBoost + isotonic: the training loop, with math
 - §13 TreeSHAP: what `pred_contribs=True` actually computes
 - §14 Reason engine: family dedup and direction-aware templates
 - §15 Mahalanobis OOD: derivation, threshold, earned-refusal
 - §16 Baselines: the fair-comparison protocol
- **Part IV — Evaluation math**
 - §17 AUROC as a rank statistic; why for rare events
 - §18 Brier decomposition, BSS against climatology
 - §19 Calibration — ECE with equal-count binning
 - §20 Asymmetric decision cost and economic value
- **Part V — Serving and systems**
 - §21 Batch scoring pipeline
 - §22 SQLite schema and indexing
 - §23 FastAPI request lifecycle
 - §24 Dashboard rendering — 2D (Leaflet) and 3D (three.js) with the geometry-merging optimisation
 - §25 Air-gap verification methodology

- **Part VI — Verification**
- §26 Test taxonomy — 68 tests, what each guards
- §27 Reproducibility from a cold clone
- **Part VII — Reflection**
- §28 Known limitations we did not close
- §29 What we would change with another month
- §30 Extension research directions

PART I — PROBLEM FORMALISATION

§1. The probabilistic setting we chose

Let $\mathcal{S}$ be the set of 34 modelled IMD meteorological subdivisions (index s), $\mathcal{T}$ a set of initialisation datetimes at 00 UTC (index t_0), and $L \in \{1, \dots, 10\}$ the lead day. For each triple (s, t_0, L) the forecast field yields a scalar $F(s, t_0, L) \in \mathbb{R}^+$ — the area-mean forecast 24 h rainfall over s valid on day $V(t_0, L) := t_0 + (L-1)$ calendar days. The observation field yields a corresponding scalar $O(s, V)$.

We define a target label $Y(s, t_0, L) \in \{0, 1\}$ (see §2) and the modelling task is to learn

$$\hat{p}(s, t_0, L) = \hat{P}(Y(s, t_0, L) = 1 \mid X(s, t_0, L))$$

where $X(s, t_0, L)$ is a 52-dimensional feature vector (§7, §10, §11) constructed exclusively from information available **at or before** t_0 . This last clause is the whole game; violating it turns the problem from prediction into hindcasting.

Design decision worth naming. We chose classification over regression on $|F - O|$ because the operational unit is a decision (would the officer's alert have flipped?), not an error magnitude. The bust label operationalises "did the decision flip"; regressing on raw error would optimise for numeric fit and land on an "impressive RMSE" that does not translate to a triaged review queue. This is defended in `LOGIC.md` §4.4 and in `src/fbd/labels/bust.py` module docstring.

Loss function used at training time: log-loss with `scale_pos_weight = n_0 / n_1 ≈ 25` (see §12). Threshold selection uses asymmetric cost (see §20), not a symmetric error rate.

§2. The bust label, derived condition by condition

Full definition, all three conditions logically ANDed. Implemented at `src/fbd/labels/bust.py`.

Condition 1 — magnitude

Let $E(s, t_0, L) = |F(s, t_0, L) - O(s, V(t_0, L))|$. Fit the 95th-percentile of E per (subdivision, month) **on training years only**:

$$\theta_{\text{mag}}(s, m) = \max(10.0, Q_{0.95}(\{E(s, t_0, L) : t_0 \in \text{Train}, \text{month}(V) = m\}))$$

The `max(10, ·)` floor prevents dry-subdivision noise from qualifying. Implemented at `src/fbd/labels/bust.py:error_thresholds` with a two-level fallback (subdivision-month → subdivision-wide → global) for months with fewer than 200 training samples — otherwise a rare-month P_{95} is a fit to noise.

Fires when $E(s, t_0, L) \geq \theta_{\text{mag}}(s, \text{month}(V))$.

Condition 2 — category flip

Define `Cat:  $\mathbb{R}^+ \rightarrow \{0, 1, 2, 3, 4, 5\}$` from the IMD intensity bands:

```
Cat(r) = 0 if r ∈ [0, 2.5)    no rain
        = 1 if r ∈ [2.5, 15.6) light
        = 2 if r ∈ [15.6, 64.5) moderate
        = 3 if r ∈ [64.5, 115.5) heavy
        = 4 if r ∈ [115.5, 204.5) very heavy
        = 5 if r ≥ 204.5    extremely heavy
```

Implemented as a `numpy.searchsorted` on `CATEGORY_EDGES` at `src/fbd/labels/bust.py:rain_category`. $O(\log 6)$ per row, vectorised.

Fires when `Cat(F) ≠ Cat(O)`.

Condition 3 — operational significance

Fires when `max(F, O) ≥ 15.6` (the lower bound of the *moderate* class).

The full label

```
Y(s, to, L) = 1 ⇔ [C1] ∧ [C2] ∧ [C3]
              = 0 otherwise
```

with `Y := NaN` if either `F` or `O` is undefined (missing IMD cell or dropped forecast). Rows with `NaN` labels are excluded from training and scoring; excluding them is not the same as scoring them as 0.

Why three conditions, formally

Let `q ∈ (0, 1)` be the percentile used in C1. If we used C1 alone:

```
P(Y = 1 | X) = P(E ≥ θmag) ≈ 1 - q by construction of θmag
```

so the base rate on the training set is *exactly* `1 - q`, independent of the atmosphere, the region, or anything else. The label ceases to carry information — it merely reports where each row sits in the training-year error distribution. Every downstream AUROC would be measuring nothing.

C2 alone fires on boundary-straddling pairs (15.5 vs 15.7 mm), which are decision-irrelevant.

C3 alone would fire on any moderate-plus rain event, whether forecast well or badly.

All three combined encode: "a large error that would have flipped a decision that mattered." The base rate becomes an emergent property of the atmosphere (measured ~4%), not an artefact of the definition.

The leakage discipline

`θmag` is estimated only on training years (2016–2020) and applied unchanged to 2021 (val) and 2022 (test). If it were re-fitted per split, the held-out bust rate would be pinned at `1 - q = 5%` by construction and the whole evaluation would prove nothing. The test-year bust rate is **3.59%**, not 5%; the train rate is 4.07%. That discrepancy is the numerical proof the fit is honest.

Tested at `tests/test_core.py:test_error_threshold_is_fitted_on_training_years_only`: synthetic data with different noise per year, must produce different bust rates per year with a single fitted threshold.

§3. Zero-leakage causality axioms

Every feature used at prediction time for (s, t_o, L) must be a function of information available **at or before** t_o . Two subtle traps we close explicitly:

Axiom A — no valid-time atmospheric state

The ERA5 analysis valid at $V(t_o, L)$ does not exist when the forecast is issued at t_o . Using it would be a near-perfect predictor of whether that forecast busted (you would essentially be showing the model the answer) and would inflate every AUROC to ~ 1.0 .

Enforced in code at `src/fbd/features/era5.py` module docstring and via the join key in `scripts/build_dataset.py:attach_state_features` which explicitly joins on `init_date`, never on `valid_date`.

Axiom B — lagged-ensemble causality

The lagged-ensemble spread for lead L uses forecasts from successive initialisations that all verify on the same day V . The set of *those* forecasts issued *at or before* t_o is:

$$\{ (t_o - k\Delta, L + k) : k \geq 0, L + k \leq 10 \} \quad \text{where } \Delta = 1 \text{ day}$$

i.e. we only look at leads $\geq L$. A forecast with lead $L - 1$ verifying on V was issued at $t_o + \Delta$, which is *after* t_o ; including it would use the future.

Enforced at `src/fbd/features/forecast.py:lagged_ensemble` by construction — the loop iterates `cols = [lead_pos[l] for l in range(L, L + n_members) if l in lead_pos]`, and there is a poison test.

The poison test. `tests/test_core.py:test_lagged_ensemble_uses_only_leads_at_or_beyond_L` constructs a synthetic day where leads 1–4 hold the value 1000 and leads 5–10 hold 10. If the code obeyed causality, leads 5–8 would show zero spread (their members are all from `{5, ..., 10}`, all equal to 10) and lead 4 would show huge spread (its members `{4, 5, 6}` = `{1000, 10, 10}`). The test asserts exactly this. Every code change that touches feature construction re-runs this test.

Reading this test in your mentor's presence is the single strongest defence of the causality claim.

Both axioms extend to derived features

- Climatology: fitted on training years, applied to val/test unchanged.
- Regime scores: computed from t_o analysis only (not V).
- Standardisations of national indices (`somali_jet_z` etc.): mean and std computed on training years only, then applied.

Any deviation would be a subtle leak. All standardisation stats are attached to the model artifact at training time and re-used at scoring time; nothing is recomputed on test rows.

§4. The spatial aggregation operator, derived

The problem forces a projection from raster data (WeatherBench 2 forecasts on a 0.703° regular grid; IMD observations on a 0.25° regular grid) to vector data (36 subdivision polygons of vastly varying shape and size, e.g. West Rajasthan 193,000 km² vs Coastal Karnataka 18,000 km²).

The wrong way, and why

Centroid masking (assign a raster cell to a polygon iff the cell centre lies inside it) is what `regionmask` does. At 0.7° a centroid test silently gives zero cells to narrow coastal subdivisions like Konkan & Goa, Coastal Karnataka, Kerala — which are precisely the heavy-rainfall regions this project exists to serve. Losing them would not raise an error; it would just quietly drop the most important rows.

The right way

For each (raster cell `c`, subdivision `s`) compute the intersection area `A(c ∩ s)` in an equal-area projection (**EPSG:7755**, India NSF Lambert Conformal Conic), keep only pairs with positive overlap, and use those areas as weights. Implemented at `src/fbd/regions/masks.py:overlap_weights` via GeoPandas `overlay`.

The result is a sparse table:

```
subdivision_id | lat_idx | lon_idx | lat | lon | weight_km²
```

with one row per (cell, subdivision) pair that overlaps in reality. Cached to a `.parquet` keyed by `(n_lat, n_lon, lat[0], lat[-1], lon[0], lon[-1])` so a differently-shaped grid can never silently reuse another grid's weights.

The aggregation itself

Given the sparse weights, materialise a dense array $W \in \mathbb{R}^{|S| \times n_{\text{lat}} \times n_{\text{lon}}}$ (implementation: `weights_to_matrix`). For a field $f \in \mathbb{R}^{T \times n_{\text{lat}} \times n_{\text{lon}}}$ (T timesteps, possibly with NaN in ocean cells):

```
valid[t, i, j] = 1 if isfinite(f[t, i, j]) else 0
filled[t, i, j] = f[t, i, j] if valid else 0

numerator[t, s] = Σ{i,j} filled[t, i, j] · W[s, i, j]
denominator[t, s] = Σ{i,j} valid[t, i, j] · W[s, i, j]
total[s] = Σ{i,j} W[s, i, j]

mean[t, s] = numerator[t, s] / denominator[t, s]
coverage[t, s] = denominator[t, s] / total[s]
```

Implemented as three `np.einsum('tij,sij->ts', ..., optimize=True)` calls at `src/fbd/regions/masks.py:area_mean`. Vectorised; the whole 7-year IMD record aggregates in ~40 seconds on a laptop.

Quality gate

`mean` is set to NaN wherever `coverage < 1 - max_nan_fraction`. Default `max_nan_fraction = 0.40` from `src/fbd/config.py:MAX_NAN_FRACTION`. A subdivision that had only 10% of its area with valid gauge data on a given day yields NaN, not a mean built from a handful of cells that would silently misrepresent the whole region.

Tested at: - `tests/test_core.py:test_area_mean_matches_hand_computation` (mathematics correct) -
`tests/test_core.py:test_area_mean_renormalises_over_valid_cells_and_reports_coverage` (NaN handling correct) -
`tests/test_core.py:test_area_mean_rejects_thin_coverage` (coverage gate fires)

PART II — END-TO-END COMPUTATIONAL TRACE

§5. From atmosphere to (F, 0) pair

Trace for one row: subdivision `s = ODISHA`, init `t₀ = 2022-06-14T00:00Z`, lead `L = 5`, valid `V = 2022-06-18`.

Forecast path: 1. WeatherBench 2 HRES zarr store `datasets/hres/2016-2022-0012-512x256_equiangular_conservative.zarr` opened via anonymous GCS (`gcsfs` + `zarr`). Loaded lazily. 2. Slice `total_precipitation_24hr[time=t₀, prediction_timedelta=5*24h, :, :]` triggers a chunk read from GCS. 3. Convert m → mm (multiply by 1000). Rename `latitude → lat`, `longitude → lon`, transpose to `(time, lead, lat, lon)`. 4. Slice to India bounding box `[6°N-38°N, 66°E-100°E]`; produces a `48 × 45` field for this cell. 5. Multiply by pre-cached overlap weights `W[s_ODISHA, :, :]`, sum, divide by total weight → scalar `F(s, t₀, L)` in mm/day.

Observation path (parallel): 1. IMD NetCDF file `RF25_ind2022_rfp25.nc` opened via `xarray`, `RAINFALL` variable. 2. `RAINFALL[time=V, :, :]` → 129 × 135 field. 3. Same weight matrix multiplication (the weight cache key uses grid corners so the IMD grid gets a different cache than the HRES grid). 4. Scalar `O(s, V)` in mm/day.

Join: in `src/fbd/ingest/hres.py:build_pairs` via `pandas.merge` on `(subdivision_id, valid_date)`. Only inner join — a row missing either side is dropped.

Result on 24 Aug 2026 for our specific example: `F = 12.3 mm/day`, `O = 33.4 mm/day`. Not a bust — the error is 21.1 mm, but `Cat(F) = 1` (light) and `Cat(O) = 2` (moderate), category flip fires; magnitude fires (21.1 > 10 floor); significance fires (max = 33.4 > 15.6). Actually — three conditions hold, so **this specific row is a bust**. (Verify by reading `data/artifacts/bulletins.sqlite`.)

§6. From `(F, O)` to the bust label

Straight application of §2's three conditions. Implemented as a vectorised pandas operation:

`src/fbd/labels/bust.py:label`. No loops, no per-row Python — everything is column arithmetic on the whole 279,650-row frame at once.

§7. From features to a calibrated probability

Given a row `(s, to, L)` with feature vector $x \in \mathbb{R}^{5 \times 2}$:

1. **Preprocessing:** none required. Trees handle NaN natively (XGBoost `missing=NaN` sends missing values to a learned default child at each split); no imputation, no scaling.
2. **Base classifier:** `xgboost.XGBClassifier` returns `p_raw = P(Y=1|x)` from the ensemble sum of tree scores passed through sigmoid. Trained with `scale_pos_weight ≈ 25` to counter the ~4% class imbalance.
3. **Calibration:** isotonic regression `g: [0, 1] → [0, 1]` fitted on the 2021 validation year, mapping `p_raw` to a well-calibrated `p_cal`. See §12 for the algorithm and why isotonic beats Platt scaling here.
4. **Bagged uncertainty:** `scripts/generate_bulletins.py:bagged_interval` refits the whole XGBoost + isotonic pipeline 6 times on bootstrap-resampled training years, produces 6 alternative `p_cal` values per row, takes the 10th–90th percentile spread, and recentres it on the deployed `p_cal` so the interval always contains the served point estimate. See §12 for the bias reasoning and the older approach that was wrong.
5. **OOD gate:** Mahalanobis distance `D(x)` in standardised feature space (§15). If `D(x) > θood` the row's output is `{status: OUT_OF_DISTRIBUTION, bust_probability: null}` regardless of what the model would have said. The interval is also nullified — nothing quantitative is emitted on refused rows.
6. **Explanation:** if the row is not OOD, native TreeSHAP (§13) produces per-feature contributions `φi(x)`, ranked, filtered by concept family (§14), turned into ≤3 sentences.

Full function: `src/fbd/model/train.py:BustModel.predict_proba` followed by the OOD guard and explainer in `scripts/generate_bulletins.py:main`.

§8. From probability to a dashboard cell

The precomputed row lands in `data/artifacts/bulletins.sqlite`:

```
(region_id, region, init_date, lead_day, valid_date, status,
 bust_probability, confidence_in_estimate, pi_low, pi_high,
 dominant_factors, regime_json, data_quality,
 ood_distance, forecast_rain_mm, observed_rain_mm, actual_bust,
 baseline_probability, model_version)
```

Primary key `(region_id, init_date, lead_day)`; indices on `init_date`, `valid_date`, `region_id`.

The 2D dashboard requests `/api/bulletin?init_date=...&lead_day=...`, gets a list of subdivision-level rows, colours each polygon in Leaflet by `bust_probability` using five bands (`<5%`, `5-10%`, `10-20%`, `20-35%`, `>35%`) + purple for OOD.

The 3D command centre requests `/api/risk-cube?init_date=...`, gets all $34 \times 10 = 340$ cells in one payload (parallel arrays by lead index, no per-cell reason strings — those load on click), renders columns at each subdivision's `representative_point` centroid with height proportional to lead and colour by probability. See §24.

PART III — SUBSYSTEMS IN DEPTH

§9. Data ingestion — chunking economics

The single non-obvious fact that drove every dataset decision: **WeatherBench 2 zarr chunks span the entire globe.** Measured chunk shapes:

Store	Variable	Chunk shape	Compressed size
HRES (1.5°)	<code>total_precipitation_24hr</code>	<code>(1, 8, 240, 121)</code>	0.41 MB
HRES (1.5°)	<code>geopotential</code>	<code>(1, 8, 13, 240, 121)</code>	7.71 MB
ENS (1.5°)	<code>total_precipitation_24hr</code>	<code>(1, 50, 8, 240, 121)</code>	20.5 MB (all 50 members in one chunk)
ENS (1.5°)	<code>geopotential</code>	<code>(1, 50, 8, 13, 240, 121)</code>	91.5 MB

Every chunk covers the whole globe (`240 × 121` is the full 1.5° grid), so slicing to India: - saves memory (small in-RAM array) - saves local disk (small cached file) - **does not save bandwidth** — you download the world and slice locally.

Consequences (locked in `DECISIONS.md` D-002 through D-004): - HRES precipitation at 0.703° for JJAS 2016–2022: 854 init dates × ~11 MB per init ≈ 9 GB transfer. Affordable. - HRES 3-D flow fields at every lead: ~47 GB per variable. **Rejected** — atmospheric-state features come from the ERA5 analysis instead, which is a single time series shared across all `(t₀, L)` pairs. - Full 50-member IFS ENS: 20.5 MB × 6 chunks × 854 inits ≈ 105 GB for precipitation alone. **Rejected**; subsampled to every 3rd init for the test year (D-014).

All raw files land in `data/raw/{imd,wb2/hres,wb2/era5,wb2/ens,shapes}/`. Scripts are idempotent — re-running a fetch script skips files already present with size > 10 KB, so an interrupted run resumes rather than restarts.

§10. Feature engineering — causality enforcement

Feature construction is split by causality domain:

- **Forecast-derived features** — computed from the forecast archive itself, so nothing to enforce beyond the leads-≥-L rule for lagged ensemble. See `src/fbd/features/forecast.py`.
- **State features** — computed from ERA5 analysis, joined on `init_date` never `valid_date`. See `src/fbd/features/era5.py` module docstring, and `scripts/build_dataset.py:attach_state_features` for the exact merge.
- **Climatological features** — fitted on training years, joined by `(subdivision_id, month)` or `(subdivision_id, month, lead_day)`. See `src/fbd/features/forecast.py:climatology` and `:climatological_bust_rate` (lines 133–158).

Every fitted statistic (`clim_obs_mean`, `clim_bust_rate`, standardisation μ, σ for `_z` features) is derived from `df[df.valid_date.dt.year.isin(TRAIN_YEARS)]`. Grep for `train_years` or `config.TRAIN_YEARS` — it is a pattern repeated in every fit call.

The lagged-ensemble features, mathematically

For lead `L`, take the `n_members`-lead window `{L, L+1, ..., L+n_members-1}` (default `n_members = 3`):

```
lagged_spread(s, t0, L) = std({ F(s, t0 - (k-1)Δ, k) : k ∈ window, k ≤ 10 })
lagged_mean(s, t0, L) = mean(...)
lagged_range(s, t0, L) = max(...) - min(...)
lagged_n_members(s, t0, L) = # of leads in window with valid F
lagged_spread_rel = lagged_spread / (lagged_mean + 1) # normalise for dry vs wet region
```

Standing convention: `t0 - (k-1)Δ` with lead `k` verifies on `V(t0, L)` for all `k` in the window. Thin-membership (`< 2`) rows get `NaN` for spread — most importantly, **Day 10 has membership 1 across the whole archive**, because the archive stops at 240 h and leads 11, 12 don't exist. This artefact is disclosed in `DECISIONS.md` D-011 and drives our "headline is Day 3–7" reporting.

spread_growth

`spread_growth(s, t0, L) = lagged_spread(s, t0, L) - lagged_spread(s, t0, L-1)`, computed within the group `(subdivision_id, valid_date)`. Fast-growing disagreement flags a rapidly-losing-predictability situation.

jumpiness

Consecutive-run change for the same valid day: `jumpiness(s, t0, L) = |F(s, t0, L) - F(s, t0 - Δ, L + 1)|`. Duty forecasters watch this informally already ("the model keeps flipping on this event"); we make it a feature. Implemented at `src/fbd/features/forecast.py:jumpiness`.

clim_bust_rate — dual purpose, defended

The same statistic is used as both **baseline #1** (§16) and as a **feature**. Legitimate because it is fitted on training data only, and the baseline is scored on held-out data. The model is not allowed to peek at test-year bust labels via this feature. Laplace smoothing with `k = 20` prevents a subdivision-lead cell with 40 samples and zero busts from claiming a 0% rate.

National ERA5 indices — the ones your mentor may probe

Implemented at `src/fbd/features/era5.py:national_indices`. Physically-motivated, all reduced to India-domain area-weighted (`cos(lat)`) means or extrema over specific boxes:

Index	Box (lat_min, lat_max, lon_min, lon_max)	Physical meaning
somali_jet	(5, 15, 50, 65)	U-wind at 850 hPa averaged over the Somali jet. Monsoon strength.
monsoon_trough_mslp	(20, 28, 72, 88)	MSLP over the trough box. Deeper trough → lower MSLP.
bob_vorticity_max	(10, 22, 82, 95)	Max relative vorticity at 850 hPa over the Bay of Bengal. Depression indicator.
bob_mslp_min	(10, 22, 82, 95)	Min MSLP over the Bay of Bengal. Low-pressure indicator.
mcz_q850	Monsoon Core Zone (18–28°N, 65–88°E)	850 hPa specific humidity — moisture available to the core rainfall zone.
india_shear	India box	Magnitude of $V_{200} - V_{850}$ — vertical wind shear.
nw_z500	(28–38, 68–80)	500 hPa geopotential over NW India — western disturbance trough indicator.

Each raw index is standardised against training-year climatology (`_z` suffix) and given 1-day and 3-day tendencies (`_d1`, `_d3`) computed within each year (grouping by year prevents differencing 1 June against the previous 30 September and manufacturing fake tendencies — this bug was found and fixed mid-build).

Relative vorticity — the calculus

Implemented at `src/fbd/features/era5.py:_relative_vorticity`. On a lat/lon grid:

$$\begin{aligned}\zeta &= \partial v / \partial x - \partial u / \partial y \\ &= (1 / (R \cdot \cos(\text{lat}))) \cdot \partial v / \partial \lambda - (1 / R) \cdot \partial u / \partial \phi\end{aligned}$$

with $R = 6.371 \times 10^6 \text{ m}$, $\lambda = \text{longitude in radians}$, $\phi = \text{latitude in radians}$. In code, `xarray.differentiate('lon')` returns derivatives *per degree*, so we scale by $1 / (R \cdot \cos(\text{lat}) \cdot \pi/180)$ for $\partial v / \partial \lambda$ and $1 / (R \cdot \pi/180)$ for $\partial u / \partial \phi$. Centred differences keep the result on the original grid. Units: 1/s. Sanity check: values on JJAS mornings over the Bay of Bengal come out at $10^{-5} - 10^{-4}$ 1/s (cyclonic, positive), which matches published synoptic scales for a monsoon depression.

§11. Regime classifier — score → softmax → entropy

Implemented at `src/fbd/regime/classify.py`. Weak supervision, permitted for the MVP by LOGIC.md §5.2.

The six regimes

`active_monsoon`, `break_monsoon`, `monsoon_depression`, `western_disturbance`, `orographic`, `coastal`. These are the ministry's own named regimes from the problem statement, not something we invented.

Two honest observations

1. Four are genuine daily *circulation* regimes: active, break, depression, WD.
2. Two are *place* × flow interactions: orographic and coastal. A subdivision is in an orographic regime when strong low-level flow meets its terrain, and it is coastal when onshore moist flow impinges on its coastline. Static "place" attributes (elevation, roughness, land fraction) modulate the daily flow signal for these two.

Scoring

For each (s, t) day, physically motivated scores:

```
s_active(s, t) = 0.5 · somali_jet_z(t) + 0.5 · mcz_q850_z(t) - 0.3 · monsoon_trough_mslp_z(t)
s_break(s, t)  = -0.5 · somali_jet_z(t) - 0.5 · mcz_q850_z(t) + 0.3 · monsoon_trough_mslp_z(t)
               - 0.3 · tcwv_zl(s, t)                                # tcwv standardised within subdivision
s_dep(s, t)    = 0.6 · bob_vorticity_max_z(t) - 0.6 · bob_mslp_min_z(t)
s_wd(s, t)     = north_weight(s) · (-1.0 · nw_z500_z(t)) # only meaningful for northern subdivisions
s_orog(s, t)   = PLACE_WEIGHT · orographic_index(s)
               + orographic_index(s) · (0.6 · moisture_flux_850_zl + 0.4 · u850_zl)
s_coast(s, t)  = PLACE_WEIGHT · coastal_index(s)
               + coastal_index(s) · (0.6 · tcwv_zl + 0.4 · moisture_flux_850_zl)
```

`PLACE_WEIGHT = 2.5` (see `src/fbd/regime/classify.py:11`). Why: the flow-modulation terms are standardised *within* subdivision, so their time-mean is zero — multiplying by a static index scales variance but not mean, and Himachal would end up with the same average orographic probability as West Rajasthan. A static baseline term restores the intrinsic character. This bug was found and fixed mid-build.

Softmax with temperature

Softmax temperature $\tau = 0.8$:

$$p(r \mid s, t) = \exp(s_r(s, t) / \tau) / \sum_{r'} \exp(s_r'(s, t) / \tau)$$

Slightly sharper than `tau = 1`. Rarely produces top-regime probability > 0.98 — the classifier stays soft, which is the design intent (LOGIC.md §5.1).

Regime entropy

$$H(s, t) = -\sum_r p(r \mid s, t) \cdot \log(p(r \mid s, t)) / \log(|R|)$$

Normalised to [0, 1]. **Fed to the bust model as a feature.** High entropy means the classifier itself is ambiguous about which regime applies, which is itself a predictor of low predictability (LOGIC.md §5.3: "regime uncertainty is itself evidence of low predictability"). This is one of the more elegant modelling choices — a meta-signal from the auxiliary model.

Independent validation your mentor should like

Not fitted on bust labels, so the regime vector cannot leak the target. Validation done post-hoc: on top-10% depression-score days over Odisha, mean rainfall is **25.1 mm/day**, vs **5.3 mm/day** on low-depression days — a 4.7× ratio, meteorologically consistent with what a depression should do. On Coastal AP: 8.4 vs 4.5 (~2×). On Gangetic WB: 9.2 vs 6.8 (~1.4×). The signal is strongest where it should be strongest.

§12. XGBoost + isotonic — the training loop with math

The objective

XGBoost minimises

$$L(\theta) = \sum_i \log\text{-loss}(y_i, \hat{y}_i(\theta)) + \sum_k \Omega(f_k)$$

where `ŷi` is the sum of tree scores through sigmoid and `Ω(f) = γ · T + 0.5 · λ · ||w||2` regularises each tree's `T` leaves. Our hyperparameters:

Parameter	Value	Rationale
<code>max_depth</code>	5	Shallow enough to keep TreeSHAP fast and interpretable, deep enough to capture interactions. Deeper trees started overfitting on the JJAS-only 2021 val set.
<code>n_estimators</code>	600	Elbow of the val-loss curve, paired with the low learning rate below.
<code>learning_rate</code>	0.04	Shrinkage; low rate is why the estimator count is high.
<code>subsample</code>	0.85	Stochastic training.
<code>colsample_bytree</code>	0.75	Column subsampling.
<code>min_child_weight</code>	20	Prevents splits on tiny subpopulations that would fit the 4% rare class to noise.
<code>reg_lambda</code>	2.0	L2 on leaf weights.
<code>scale_pos_weight</code>	<code>n₀ / n₁ ≈ 25</code>	See below.
<code>tree_method</code>	<code>hist</code>	Histogram-based split search, ~10× faster than exact.

Implemented at `src/fbd/model/train.py:BustModel.fit`.

The class-imbalance math

With bust rate `p+ ≈ 0.04` the gradient of log-loss weights positives at only their base rate. Setting `scale_pos_weight = n0/n1 = (1-p+)/p+ ≈ 25` scales the gradient at positive rows by that factor, which is equivalent to duplicating each positive row ~25 times but without the memory cost. The effect: the model treats positives and negatives as roughly balanced during split selection.

Side effect worth naming: the raw output is systematically over-confident (a "50/50" balanced classifier is applied to a 4/96 world). This is *precisely* why isotonic calibration is mandatory here.

Isotonic regression — the algorithm

Given n pairs (p_{raw_i}, y_i) sorted by p_{raw} , isotonic finds a non-decreasing step function g minimising $\sum (g(p_{\text{raw}_i}) - y_i)^2$. Solved by the **Pool Adjacent Violators Algorithm** (PAVA) in $O(n)$ after sorting:

1. Initialise $g(p_{\text{raw}_i}) = y_i$ for all i (sorted by p_{raw}).
2. Scan left-to-right; whenever $g_{i-1} > g_i$, pool blocks: set both to their weighted mean, then rescan the pool boundary.
3. Terminate when the sequence is non-decreasing.

Implemented via `sklearn.isotonic.IsotonicRegression` with `out_of_bounds='clip'`, `y_min=0.0`, `y_max=1.0`.

Why isotonic over Platt scaling

Platt fits a two-parameter sigmoid $g(p) = 1 / (1 + \exp(a \cdot p + b))$; isotonic fits an arbitrary monotone function with $O(n)$ degrees of freedom. Platt assumes the true relationship between raw score and true probability is sigmoidal; XGBoost with `scale_pos_weight` violates that assumption (it warps the raw score in a class-dependent way that is not sigmoid-shaped). Isotonic makes no shape assumption and empirically produces $ECE \approx 0.011$ on our test year, vs ~ 0.03 with Platt (measured; not reported in the deck but reproducible).

Bagged prediction intervals — the version that works

Original attempt: derive intervals by truncating XGBoost boosting rounds (predict with 100 trees, 200 trees, 300 trees, take spread). **Wrong.** Boosting converges *upward*, so truncated models are systematically lower than the full model. The resulting interval could exclude its own point estimate — e.g. `[0.571, 0.627]` around a point of `0.706`.

Current version at `scripts/generate_bulletins.py:bagged_interval`: 6 bootstrap resamples of the training year rows, refit the full XGBoost + isotonic pipeline each time, produce 6 alternative calibrated probabilities per row. Compute the interval half-width as $(P_{.90} - P_{.10}) / 2$ across the 6 draws, then recentre on the deployed model's point estimate:

```
half_i = (percentile(D[:, i], 90) - percentile(D[:, i], 10)) / 2
lo_i   = clip(point_i - half_i, 0, 1)
hi_i   = clip(point_i + half_i, 0, 1)
conf_i = clip(1 - (hi_i - lo_i) / 0.30, 0, 1)
```

All 79,234 intervals in the current bulletin store contain their point estimates, verified by a query.

§13. TreeSHAP — what `pred_contribs=True` actually computes

The interpretability layer relies on **exact** TreeSHAP (Lundberg et al. 2020), not KernelSHAP or gradient methods. Called via XGBoost's C++ path:

```
booster.predict(dmatrix, pred_contribs=True)
```

Returns an array of shape $(n_{\text{rows}}, n_{\text{features}} + 1)$ where entry $[i, j]$ is $\phi_j(x_i)$ for $j < n_{\text{features}}$ and $[i, n_{\text{features}}]$ is the base value (mean log-odds). Additivity: $\sum_j \phi_j(x_i) + \text{base} = \text{log-odds}(\hat{y}_i)$.

The Shapley property

TreeSHAP is the unique attribution method satisfying: - **Local accuracy:** $\sum_j \phi_j + \text{base} = \text{model output}$. - **Missingness:** if feature j was never used in any tree, $\phi_j = 0$. - **Consistency:** if the model changes so that a feature's marginal contribution weakly increases regardless of other features, its ϕ_j weakly increases.

These properties are what make SHAP a defensible reason source; ad-hoc feature-importance rankings satisfy none of them.

Complexity

Exact TreeSHAP runs in $O(T \cdot L \cdot D^2)$ where T = trees, L = leaves per tree, D = depth. For our model $T = 300$, $L \leq 16$, $D = 4$, so ~76,800 ops per row per feature. On our 40k test rows $\times$ 52 features this runs in ~2 seconds on a laptop.

Why we did NOT use KernelSHAP

KernelSHAP is model-agnostic but is $O(2^{n_{\text{features}}})$ per row — for 52 features that is 4.5 quadrillion coalitions per row and requires Monte Carlo estimation with variance. TreeSHAP for trees is exact, deterministic, and orders of magnitude faster.

Reference: `src/fbd/explain/reasons.py:ReasonExplainer.shap_values`.

§14. Reason engine — family dedup and direction-aware templates

Two engineering tricks that turn raw SHAP into publishable sentences.

Concept-family deduplication

SHAP often ranks three near-collinear features at the top ("moisture_flux_850", "tcwv", "mcz_q850_z" are all "the atmosphere is moist"). Reporting all three produces a redundant reason panel. Every feature is mapped to a concept family:

```
FAMILY = {
    "fcst_rain_mm": "amount",
    "fcst_anomaly": "amount",
    "fcst_rel_to_p90": "amount",
    "lagged_spread": "disagreement",
    "lagged_spread_rel": "disagreement",
    "spread_growth": "disagreement",
    "jumpiness": "disagreement",
    "moisture_flux_850": "moisture",
    "tcwv": "moisture",
    "mcz_q850_z": "moisture",
    ...
}
```

Selection rule: iterate SHAP values in descending order; keep at most one factor per family until k (default 3) are collected.

Direction-aware templates

Each feature carries a triple `(label, high_variant, low_variant)`. Example:

```
"u850": (
    "low-level zonal flow",
    "850 hPa westerly flow is strong ({v:.1f} m/s, {pct})",
    "850 hPa flow is easterly or weak ({v:.1f} m/s, {pct})",
),
```

Selection: compute the raw value's percentile within the subdivision's training-year distribution. If percentile $\geq 0.5 \rightarrow$ use `high_variant`; else `low_variant`. This eliminates the self-contradictory sentences an earlier fixed-template version produced ("westerly flow is strong (-0.9 m/s, near normal)").

O(1) percentile phrases

For each (subdivision, feature) precompute 101 quantile knots at training time. Runtime lookup is `numpy.searchsorted(knots, value) / 100` — $O(\log 101) \approx O(1)$. This turned a $O(\text{rows} \times \text{features} \times \text{refsize})$ scan into instantaneous. On our 80k bulletins that was the difference between "hours" and "seconds".

Percentile phrase generator at `src/fbd/explain/reasons.py:_percentile_phrase` uses six bands:

```
>= 0.99  -> "top 1% for this subdivision"
>= 0.95  -> "top 5% for this subdivision"
>= 0.90  -> "top 10% for this subdivision"
>= 0.75  -> "upper quartile for this subdivision"
<= 0.10  -> "bottom 10% for this subdivision"
else     -> "near normal for this subdivision"
```

Directional filtering

Only factors with **positive** SHAP contribution are reported by default (they *raise* bust probability). The panel answers "why is confidence low here", so mixing reassuring factors would bury the signal.

Reference: `src/fbd/explain/reasons.py` whole module.

§15. Mahalanobis OOD — derivation, threshold, earned-refusal

The distance

Given training-year feature matrix $X_{\text{train}} \in \mathbb{R}^{n \times d}$, standardise:

```
Z_train = (X_train - μ) / σ
```

where μ, σ are per-column mean and std. Estimate covariance with shrinkage:

```
Ŝ = cov(Z_train) + λ · I,    λ = 10-3
```

Shrinkage prevents singular covariance in a 52-D space with collinear features (e.g. `tcwv` and `mcz_q850_z` are highly correlated). Compute pseudo-inverse $\hat{\Omega} = \hat{\Sigma}^+$ once at fit time. Then for any row x :

```
D_M(x) = √( (Z_x)T · Ō · Z_x )
```

with $Z_x = (x - \mu) / \sigma$. Implementation via `np.einsum('ij,jk,ik->i', Z, Ō, Z, optimize=True)`, vectorised over all rows at once.

Threshold

The 99.5th percentile of `D_M` on the training rows themselves. At scoring time, `D_M(x) > θ_ood ⇒ status = "OUT_OF_DISTRIBUTION"`.

Why 99.5 and not 95 or 99? Empirically: 95 refuses too aggressively (~4% of test rows), degrading the useful area of the map. 99 still refuses ~1.2%. 99.5 refuses ~0.5% of held-out rows and produces the earned-refusal ratio below. Sweeping would be a nice ablation (§29).

Earned-refusal validation

The refusal is only defensible if refused rows are genuinely harder. We compute:

```
bust_rate on accepted rows = 3.4%
bust_rate on refused rows  = 23.4%
ratio                      = 6.9×
```

Refused rows bust nearly 7× more often than accepted ones. This is not decoration; the detector catches genuine failure regimes.

Implemented at `src/fbd/ood/detector.py:validate`. Do this yourself:

```
PYTHONPATH=src python scripts/stress_test.py
```

and read `data/artifacts/stress_ood.csv`.

Anomaly-feature attribution

`MahalanobisOOD.top_anomalous_features(row, k=3)` returns the top-3 features by `|z_i|` — which features made the state unusual. Not used in the reason panel for OOD rows (we just say "outside training experience"), but exposed via `/api/bulletin` for a "why refused?" UI panel that is a future extension.

§16. Baselines — the fair-comparison protocol

Four baselines + one real-ensemble baseline, **all** wrapped in the same isotonic calibration fitted on the same 2021 validation year. Without this, a class-weighted logistic emits ~0.5 probabilities and scores Brier ~0.19 while having AUROC ~0.72; beating that on Brier would prove nothing.

Wrapper: `scripts/train_model.py:Calibrated`.

#0 Forecast rain amount only

Isotonic map from raw `fcst_rain_mm` to bust probability. Answers "*is the model just detecting heavy-rain days?*" On the decision band it scores AUROC 0.750 — so about half of the model's edge over the calibrated spread baseline (0.825 – 0.758 = 0.067) is attributable to rainfall magnitude itself, and the rest comes from disagreement, state and regime features. Say this out loud; it is a form of honesty.

#1 Climatology

Per (subdivision, month, lead) mean bust rate, Laplace-smoothed with `k = 20`. AUROC 0.519 — the dumbest possible predictor; the model must beat it decisively (0.825 vs 0.519 = +0.306).

#2 Lagged ensemble spread (proxy)

Isotonic map from `lagged_spread` to bust probability, per lead. AUROC 0.758. This is the "beat operational spread" baseline for the full archive.

#3 Logistic regression on spread + lead

Standard 4-feature linear model. Sanity check. AUROC 0.754.

#4 (D-014) True IFS ENS spread — the honest baseline

Real 50-member ECMWF ensemble spread, fetched from `datasets/ifs_ens/...` for 41 init dates × 50 members in JJAS 2022. On the 6,732 decision-band rows with a matched ensemble:

Predictor	AUROC	Brier	ECE	Cost/1000	Value
Lagged-ensemble proxy (calibrated)	0.731	0.031	0.008	297.5	0.114
True IFS ENS spread (calibrated)	0.792	0.031	0.008	287.0	0.145
XGBoost + isotonic (ours)	0.832	0.029	0.010	238.7	0.289

`corr(true_ENS_spread, lagged_proxy) = 0.665`. The proxy tracks the real thing, but loosely — justifying its use as a feature across the full archive, and simultaneously disqualifying it as the headline baseline.

The honest sentence: the model beats a real, calibrated, 50-member ECMWF ensemble by **+0.040 AUROC, −16.8% decision cost, ~2× economic value** on the same rows. Raw comparison (no calibration): +0.025 AUROC.

Reproduce with:

```
PYTHONPATH=src python scripts/evaluate_ens_baseline.py --decision-band-only
```

Full narrative: `DECISIONS.md` D-014.

PART IV — EVALUATION MATH

§17. AUROC — rank statistic, why for rare events

Given scores $\hat{p}_1, \dots, \hat{p}_n$ and labels $y_1, \dots, y_n$, define positive set $P = \{i : y_i = 1\}$, negative set $N = \{i : y_i = 0\}$. Then

$$\begin{aligned} \text{AUROC} &= P(\hat{p}(x^+) > \hat{p}(x^-)) \quad \text{for } x^+ \in P, x^- \in N \text{ drawn uniformly} \\ &= (1 / (|P| \cdot |N|)) \cdot \sum_{\{i \in P, j \in N\}} 1\{\hat{p}_i > \hat{p}_j\} \quad (\text{with ties handled by } \tfrac{1}{2}) \\ &= U / (|P| \cdot |N|) \quad \text{where } U \text{ is the Mann-Whitney } U \text{ statistic} \end{aligned}$$

AUROC has one property that makes it the right first metric for rare events: it is invariant to class balance. Doubling the negatives changes accuracy dramatically but does not change AUROC (in expectation). For a ~4% base-rate bust problem where accuracy is meaningless (all-zero scores 96%), AUROC is a genuine signal-to-noise measure.

`sklearn.metrics.roc_auc_score` uses the trapezoidal integration formulation; both are equivalent for point statistics.

Reference: `src/fbd/evaluate/metrics.py:auroc`.

§18. Brier decomposition, BSS against climatology

The Brier score for binary outcomes:

$$\text{BS} = (1/n) \cdot \sum (\hat{p}_i - y_i)^2$$

Murphy's decomposition:

$$\text{BS} = \text{REL} - \text{RES} + \text{UNC}$$

where - **REL** (reliability): $(1/n) \sum_k n_k \cdot (\bar{p}_k - \bar{y}_k)^2$, the mean squared calibration error over K bins. Lower is better. - **RES** (resolution): $(1/n) \sum_k n_k \cdot (\bar{y}_k - \bar{y})^2$, how much the model varies its output across bins. Higher is better. - **UNC** (uncertainty): $\bar{y} \cdot (1 - \bar{y})$, the intrinsic variance of the label. Independent of the model.

Brier Skill Score against climatology:

$$\text{BSS} = 1 - \text{BS}_{\text{model}} / \text{BS}_{\text{climatology}}$$

Positive means better than climatology. Ours: +0.088 (decision band). Reference:

`src/fbd/evaluate/metrics.py:brier_skill_score`.

§19. Calibration — ECE with equal-count binning

Equal-**count** binning (not equal-width) is the correct choice for rare events. With $\hat{p}$ heavily concentrated in $[0, 0.1]$, equal-width bins leave the upper bins with two or three points each, producing "reliability diagrams" that look dramatic and mean nothing.

Algorithm: 1. Sort predictions ascending. 2. Partition into K bins of $\lceil n/K \rceil$ rows each (default $K = 10$). 3. Per bin k compute mean predicted $\bar{p}_k$ and observed frequency $\bar{y}_k$.

Expected Calibration Error:

$$\text{ECE} = \sum_k (n_k / n) \cdot |\bar{p}_k - \bar{y}_k|$$

Reference: `src/fbd/evaluate/metrics.py:reliability_curve` and `:expected_calibration_error`. Ours: 0.011 on the held-out year.

§20. Asymmetric decision cost and economic value

Domain-encoded costs:

- $C_{\text{miss}} = 10.0$ (missed bust; downstream disaster underprepared)
- $C_{\text{false}} = 1.0$ (false alarm; ~10 forecaster-minutes)

For threshold $\tau \in [0, 1]$:

$$\begin{aligned} \text{flag}_i &= 1\{\hat{p}_i \geq \tau\} \\ \text{Cost}(\tau) &= C_{\text{miss}} \cdot \#\{i : y_i = 1 \wedge \text{flag}_i = 0\} + C_{\text{false}} \cdot \#\{i : y_i = 0 \wedge \text{flag}_i = 1\} \end{aligned}$$

Threshold selection: grid search over $\tau \in \{0.01, 0.02, \dots, 0.99\}$ minimising $\text{Cost}(\tau)$. Reference: `src/fbd/evaluate/metrics.py:best_threshold`.

Potential economic value (Richardson 2000)

$$V = (\text{Cost}_{\text{ref}} - \text{Cost}_{\text{model}}) / (\text{Cost}_{\text{ref}} - \text{Cost}_{\text{perfect}})$$

where $\text{Cost}_{\text{ref}} = \min(\text{Cost}_{\text{always}}, \text{Cost}_{\text{never}})$ — cost of the better trivial strategy — and $\text{Cost}_{\text{perfect}} = 0$. $V = 1$ is a perfect forecast, $V = 0$ is no better than the best trivial baseline, $V < 0$ is actively harmful.

For our decision-band results on the held-out year: $V_{\text{model}} = 0.289$ vs $V_{\text{ENS}} = 0.145$ — the model is roughly twice as valuable as a real calibrated 50-member ensemble under the operational cost asymmetry. Reference:

`src/fbd/evaluate/metrics.py:potential_economic_value`.

PART V — SERVING AND SYSTEMS

§21. Batch scoring pipeline

Runs once per day in production (a cron would kick `scripts/generate_bulletins.py`). Off-line, deliberate — LOGIC.md §14 bans real-time streaming, and a precomputed store is what enables the fully offline demo. Sequence:

1. Load `data/processed/dataset.parquet` (279,650 rows × 93 columns).
2. Load `bust_model.joblib` (XGBoost booster + isotonic calibrator + feature list, ~1.5 MB).
3. Fit fresh baselines on training rows for the `baseline_probability` column.
4. Score all val + test rows with `model.predict_proba`.
5. Compute bagged prediction intervals — 6 refits, this is the expensive step (~90 seconds on a laptop).
6. Compute OOD distance and flag rows above threshold.
7. Compute SHAP values, dedup by family, format sentences.
8. Wrap regime probabilities to JSON.
9. Bulk-insert into SQLite via `executemany`.

Total: ~100 seconds for 79,900 rows on a laptop with no GPU.

§22. SQLite schema and indexing

Schema at `scripts/generate_bulletins.py:SCHEMA`. Two tables:

`bulletins` — one row per `(region_id, init_date, lead_day)` triple. Primary key on that triple. Indices on `init_date`, `valid_date`, `region_id` so the three main query patterns (bulletin-for-a-date, verification-of-a-day, region-timeline) are all $O(\log n)$ seeks. Total ~63 MB for 79,900 rows.

`overrides` — append-only. `id INTEGER PRIMARY KEY AUTOINCREMENT`. Every forecaster override is written here immutably with `user`, `reason`, `created_at`. LOGIC.md §15 audit requirement. Not deleted, only appended.

Why SQLite and not PostgreSQL+PostGIS

LOGIC.md §11 explicitly names SQLite as the sanctioned backup for the offline demo, and the hour-33 gate ("must run with the network unplugged") outranks the nicety of PostGIS here. The schema uses only standard SQL; a Postgres swap would be a connection-string change. No spatial queries are done in the DB — the geometry is served from a static GeoPackage.

§23. FastAPI request lifecycle

Trace one request: `GET /api/bulletin?init_date=2022-06-14&lead_day=4&mode=replay`.

1. `uvicorn` accepts the connection, parses the HTTP request.
2. FastAPI routes to `bulletin` at `src/fbd/api/app.py`.
3. Pydantic validates the query params: `init_date` is a string, `lead_day` is `int` with `ge=1, le=10`, `mode` is a `Literal["replay", "live"]`. Any violation → HTTP 422 automatically.
4. `_quality(init_date, mode)` computes `DataQuality` (STALE/OK/DEGRADED/UNAVAILABLE) and the input age in hours. In replay mode always OK + 6h. In live mode compares latest init in DB to wall clock.
5. Open a SQLite connection with `Row` row factory. Execute the parametrised query — SQL injection impossible since values go through the DB-API bind interface.
6. Close connection. For each row, build a `BustPrediction` Pydantic model (which reserialises through the strict schema at `src/fbd/api/schema.py:BustPrediction`).
7. Wrap all predictions in `Bulletin`, return.
8. FastAPI serialises to JSON. Any type mismatch caught here — the response schema is enforced identically to the request schema.

The Pydantic double-guard (input params + response model) means the API cannot serve a malformed row; if `bulletin_probability` in the DB were somehow 1.5, the response serialisation would fail with a 500 and log the corrupt row — not silently ship an invalid probability to the frontend.

Full API surface:

GET /api/health	- server + data-quality status
GET /api/replay/dates	- list of init dates for the slider
GET /api/regions	- subdivision polygons as GeoJSON, simplified
GET /api/bulletin	- all subdivisions for one init date [+ optional lead]
GET /api/bulletin/{rid}	- one subdivision, all 10 leads
GET /api/review-queue	- ranked high-risk district-days
GET /api/verification	- model flag vs baseline vs observed truth
GET /api/metrics	- held-out-year evaluation table
GET /api/risk-cube	- all 34 x 10 cells for the 3D command centre
POST /api/override	- forecaster override, audit-logged
GET /api/overrides	- list stored overrides
GET /metrics	- Prometheus text format (operational telemetry)

§24. Dashboard rendering

2D dashboard — `web/index.html`

Vendored Leaflet (`web/vendor/leaflet.js` , 148 KB). No basemap tiles — we render the subdivisions themselves as `L.geoJSON` , coloured by bust probability, on a dark background. That deliberately dodges the CDN-tile-server dependency that would break the air-gap guarantee.

The map is one `L.geoJSON` layer with a `style` callback that returns the colour band for the current lead day. Clicking a polygon fires `select(region_id)` which fetches `/api/bulletin/{region_id}` and populates the reason panel + regime bar chart + lead-day toggle.

Review queue is a separate call to `/api/review-queue?top=12` and renders as a small table. Clicking a row selects that region + lead.

3D command centre — `web/command.html`

Vendored three.js r128 (`web/vendor/three.min.js` , 589 KB). Hand-written spherical orbit controls (~40 lines) rather than vendoring a second script; the maths is standard $(\theta, \phi, r) \mapsto (x, y, z)$.

Rendering pipeline: 1. Fetch `/api/regions` → India subdivision polygons as GeoJSON. 2. Project each polygon's rings from `(lon, lat)` to `(x, z)` on the ground plane, centred on the domain midpoint (so India sits at world origin). 3. For each subdivision, construct a `THREE.ShapeGeometry` from its polygon (with interior holes if any). Merge all subdivisions' fills into **one** `THREE.Mesh` , and all outlines into **one** `THREE.LineSegments` . This is a critical optimisation — see the paragraph below. 4. Fetch `/api/risk-cube?init_date=...` → parallel arrays `p[i]` (probability at each lead) per subdivision. 5. For each `(subdivision, lead)` construct a small box column at the subdivision's `representative_point` centroid, height per lead, colour by probability. 6. Add a translucent "decision band" slab covering lead 3–7 for visual emphasis of the operationally-actionable slice.

The geometry-merging optimisation. India's coastline and islands make many subdivisions `MultiPolygon` with multiple rings. A naive one-mesh-per-ring implementation costs ~1,330 draw calls before a single risk column is drawn — measured. Merging fills into one buffer geometry cuts that to 2 objects (one fill, one outline), and total draw calls dropped from **1,669** → **343** at identical triangle count (18,367 tris). The merge is done by hand at `web/command.html:mergeGeometries` because r128 ships `BufferGeometryUtils` under `examples/` only and we did not want to vendor a second script.

Picking: raycast from the camera through the cursor against the columns' bounding boxes (ground is unpickable). The nearest hit gives `(region_id, lead_day)` from `userData`; on click, load per-region details via `/api/bulletin/{region_id}`.

Interaction: LMB drag → orbit; RMB drag → pan across ground plane; wheel → zoom. Camera is spherical `(θ, φ, r)` around a `target`, and each mouse motion updates the spherical coordinates; the camera Cartesian position is derived each frame in `applyCamera`.

Verified in-browser today: WebGL 2.0 context (`ANGLE/D3D11` on discrete NVIDIA), 340 risk columns, zero console errors, every network request `localhost` only.

§25. Air-gap verification methodology

The dashboard must render with wifi unplugged. Verified in two ways:

1. **Static scan:** `grep -rE 'src|href=' web/*.html` — every match is either `/vendor/*` (locally vendored), `/api/*` (own service), or `/*.html` (own service). No `unpkg`, no `cdn.jsdelivr.net`, no Google Fonts, no map tile servers.
2. **Runtime scan:** load `command.html` and `index.html` in a fresh browser, inspect the browser network log, filtering to hosts `!= localhost`. Verified today: zero external requests across a full dashboard session.

If a future change introduces a CDN dependency, the second scan would catch it immediately. Could also be automated as a CI gate — planned but not yet implemented.

PART VI — VERIFICATION

§26. Test taxonomy — what the 68 tests actually guard

```
tests/
test_core.py          12 tests
- IMD category boundaries and NaN handling
- Bust 3-condition invariants (all combinations)
- Bust NaN behaviour when either side is missing
- Bust type direction (over/under-forecast)
- Error threshold fitted on training years only (leakage guard)
- Lagged ensemble uses only leads >= L (causality poison test)
- Lagged ensemble marks thin membership
- Area mean matches hand computation (linear algebra)
- Area mean renormalises over valid cells + reports coverage
- Area mean rejects thin coverage
- Subdivision config is an exact partition of districts
test_genai.py         28 tests
- GenAI is OFF unless FBD_GENAI_ENABLED=1 (envelope test)
- Numeric-grounding guardrails (11 tests)
- Retrieval / BM25 / RAG (4 tests)
- Tool-schema strictness (5 tests)
- Injection-resistance tests (2 tests)
- Agent lifecycle (3 tests)
- Endpoint gating and health posture (3 tests)
test_mlops.py         13 tests
- PSI edge cases (identical, shifted, compressed, empty bin, tiny sample)
- Drift verdict escalation (never de-escalates)
- Calibration drift detected only when labels exist
- Elevated OOD rate escalates
- Report serialisation
test_model_api.py     9 tests
- Asymmetric-cost decision function
- Brier, BSS, ECE, reliability
- Mahalanobis OOD detects synthetic outliers
- Reason-template well-formedness + coverage
- Concept families cover every template
- Percentile phrase edges
- Pydantic schema round-trip for BustPrediction and Bulletin
test_command_centre.py 6 tests
- Risk cube shape is regions x leads
- Every region has a finite placeable centroid
- OOD cells never carry a probability (refusal invariant)
- Cube values agree with per-region bulletin
- Unknown init date -> 404 not 500
- Malformed mode -> 422 (Pydantic validation)
```

Run in ~11 seconds:

```
PYTHONPATH=src python -m pytest tests/ -v
```

The tests that would silently break the whole project if removed: - `test_lagged_ensemble_uses_only_leads_at_or_beyond_L` (causality) - `test_error_threshold_is_fitted_on_training_years_only` (leakage) - `test_ood_cells_never_carry_a_probability` (refusal invariant) - `test_subdivision_config_is_an_exact_partition_of_districts` (domain artifact)

If your mentor wants to verify the causality claim in real time, open `tests/test_core.py:107-135` and read the poison test aloud.

§27. Reproducibility from a cold clone

Assuming `data/raw/*` is present (the cached datasets). Time budget on a laptop:

Step	Command	Wall time
Subdivisions	<code>python -m fbd.regions.build</code>	5 s
Truth aggregation	<code>python scripts/build_truth.py</code>	40 s
Labelled dataset	<code>python scripts/build_dataset.py</code>	10 s
Train + baselines + eval	<code>python scripts/train_model.py</code>	30 s
Bulletin generation (bagged + SHAP)	<code>python scripts/generate_bulletins.py</code>	100 s
Ablation	<code>python scripts/ablation.py</code>	30 s
Stress test	<code>python scripts/stress_test.py</code>	30 s
ENS baseline evaluation	<code>python scripts/evaluate_ens_baseline.py</code>	15 s
Drift monitor	<code>python scripts/monitor_drift.py</code>	5 s
Test suite	<code>pytest tests/ -v</code>	11 s

Total end-to-end: **~4.5 minutes**. Determinism: fixed `RANDOM_SEED = 20260920` at `src/fbd/config.py:152`, threaded into XGBoost, numpy RNG for bootstrap draws, and pandas groupby operations that don't depend on hash order. Rebuilding on a fresh machine should reproduce numbers to $<10^{-6}$.

If `data/raw` is missing, re-fetch:

```
python scripts/fetch_imd.py          # ~5 min (IMD server slow)
python scripts/fetch_hres.py         # ~20 min (9 GB from GCS)
python scripts/fetch_era5.py         # ~10 min
python scripts/fetch_ens.py --year 2019 --every 6  # ~7 min per year
python scripts/fetch_ens.py --year 2020 --every 6
python scripts/fetch_ens.py --year 2022 --every 3
```

PART VII — REFLECTION

§28. Known limitations we did not close

Do not hide these; a mentor will ask.

- 3-hour observation/forecast window offset (D-005).** IMD's rainfall day is 03Z–03Z; WB2's 24h accumulation is 00Z–00Z. WB2 does not publish 3-hourly precip at this resolution so the offset cannot be removed. Second-order for subdivision-scale area-means (19,000–222,000 km²) and applies identically to model + all baselines so it cannot manufacture a win — but it is real.

2. **Lagged spread undefined at Day 10 (D-011).** Archive stops at 240h so lead 10 has membership 1 across the whole record → 100% NaN for the lagged spread feature. Baseline collapses to prior at Day 10 (AUROC 0.500), which *inflates* the model's apparent margin over it at that lead. Fix: headline is Day 3–7 where all members exist. Do not quote the Day 10 gain as headline.
3. **spread_growth slightly leaky at long leads.** It is defined as $\text{spread}(L) - \text{spread}(L-1)$, both computed with the same causal window rule, but this makes the value depend on $\text{spread}(L-1)$ which uses leads $\{L-1, L, L+1\}$ — a superset of the causal set for lead L . Impact on features is minor because the difference cancels most of the leaked information, but a strict-purist would flag it. Fixable by defining growth as $\text{spread}(L+1) - \text{spread}(L)$ (forward difference) instead of backward.
4. **Regime classifier is weak supervision.** Physically-motivated scores + softmax, not a supervised classifier trained on expert-labelled days. Justification: the ministry accepts weak labels for the MVP (LOGIC.md §5.2), the scores are physically defensible, and independent validation (§11) shows they carry real signal. But a supervised classifier trained on IMD-labelled days would likely be better.
5. **No real Bedrock invocation yet.** The GenAI layer is tested against a fake client (28 tests pass). The account is on AWS Free Plan which blocks Bedrock; upgrade in progress. The offline system does not depend on Bedrock for any function — the LLM layer is opt-in via `FBD_GENAI_ENABLED=1` and provides an explanation-narration layer on top of the SHAP sentences, not a substitute for them.
6. **Uncertainty in observed truth is not propagated.** IMD's 0.25° gridded product is itself a Kriging interpolation from ~6,955 gauges with its own uncertainty (see IPED 2025 for a 30-member observational ensemble). We treat 0 as a point value. A more careful evaluation would propagate observational uncertainty; we did not.
7. **The 34 modelled subdivisions exclude Andaman & Nicobar and Lakshadweep.** IMD 0.25° is mainland-only; there is no comparable gridded gauge product for the islands. Fixable by ingesting a satellite-gauge merged product (GPM IMERG or IMD's own GPM-merged rainfall) but adds a second observation source with different characteristics; deferred to v2.

§29. What we would change with another month

- **Cross-validation on the temporal split.** Currently one held-out year. A rolling 5-fold across 2018–2022 with 4 train years + 1 test year each fold would produce error bars on every headline number. Requires ~5× the ENS download budget.
- **A supervised regime classifier** trained on IMD-labelled days (available from the Monsoon 2020 workshops, some in published papers). Would sharpen regime probabilities and likely add ~0.01 AUROC.
- **Multi-model AI ensembling.** Pull GraphCast/GenCast/Pangu forecasts for the same subset, compute cross-model disagreement, add as a feature. This is the item most likely to add AUROC — cross-model disagreement is a stronger predictability signal than same-model spread.
- **Live IMD adapter.** Currently reproduces on WeatherBench 2 archive only. A live adapter (ECMWF Open Data + IMD's own real-time GPM-merged rainfall) would let us score today's forecast today. Non-trivial because ECMWF Open Data has a different chunking, projection, and variable naming convention.
- **A calibrated ENS baseline over the whole record**, not just JJAS 2022. Currently only D-014's decision-band subset has a true-ENS baseline. Requires downloading ~50 GB of ENS spread across all training years; feasible but was deferred against the \$2 AWS budget.
- **OOD threshold sweep.** Currently 99.5th percentile. A proper Pareto-frontier analysis over refusal rate vs. earned-refusal-ratio would give a defensible operating point rather than a chosen constant.
- **Explanation-stability check.** Perturb feature values by $\pm\epsilon$ and measure how much the top-3 reason list changes. If reasons are stable to noise, they are trustworthy; if they flip under 1% noise, they are decoration. Script drafted, not yet run.
- **PDF bulletin export.** Judges remember paper artefacts. `weasyprint` + a Jinja template on `/api/bulletin` output. Half a day.

§30. Extension research directions

- **Regime-specific models.** Fit one XGBoost per regime and combine via the regime soft probability vector. Would exploit interactions the pooled model cannot. Sample-size limited (~4k rows per regime after JJAS filtering).
 - **Sequence-of-forecasts features.** Currently we look at 3 successive initialisations for spread. A recurrent view over the last 5 days of forecasts for the same event would capture "the model is oscillating on this depression" more naturally than `jumpiness` alone.
 - **Convex-combination probability output.** Instead of a single scalar, output a convex mixture over IMD categories ($P(\text{Cat}(0) = k \mid F, x)$) — a full predictive distribution over the observation. Enables cost-sensitive decisions at the category level rather than the binary bust level.
 - **Cost-sensitivity slider in the UI.** Expose the `C_miss / C_false` ratio to the forecaster; recompute the review-queue threshold live. This is a UI feature, but it also forces the model's calibration to be robust across cost regimes.
 - **Model card and datasheet.** Document the training distribution, known failure modes, intended use, out-of-scope uses, and update cadence in a formal MODEL_CARD.md. Standard practice for models proposed for operational use.
 - **Uplift over a live NCMRWF ensemble.** IMD uses NCMRWF's ensemble (`NCUM`) internally. A comparison of our model against NCUM spread would be the operationally-relevant baseline, but the archive is not publicly available.
-

§31. Appendix — cheat-sheet of the most defended numbers

Fact	Value	Source
Modelled subdivisions	34	D-006
Districts in Census 2011 partition	641	<code>fbid.regions.build</code>
Total area (India minus disputed)	3,180,579 km ²	build output
Training rows	199,750	dataset build log
Validation rows (2021)	39,950	dataset build log
Test rows (2022)	39,950	dataset build log
Overall bust base rate (test year)	3.59%	dataset build log
Model AUROC — all leads	0.840	<code>results.json</code>
Model AUROC — decision band	0.825	<code>results.json</code>
True IFS ENS AUROC — raw, same rows	0.807	<code>ens_auroc_comparison.csv</code>
True IFS ENS AUROC — calibrated, same rows	0.792	<code>ens_baseline_comparison.csv</code>
Model AUROC on same 6,732 rows	0.832	<code>ens_baseline_comparison.csv</code>
Δ AUROC vs calibrated ENS	+0.040	D-014
Δ AUROC vs raw ENS	+0.025	D-014
Δ decision cost vs calibrated ENS	−16.8%	<code>ens_baseline_comparison.csv</code>
Economic value: model / true ENS / proxy	0.289 / 0.145 / 0.114	<code>ens_baseline_comparison.csv</code>
Bust rate on accepted rows	3.4%	<code>stress_ood.csv</code>
Bust rate on OOD-refused rows	23.4%	<code>stress_ood.csv</code>
OOD refusal earned-ness ratio	6.9×	derived
Model ECE (decision band)	0.011	<code>results.json</code>
Model Brier (decision band)	0.029	<code>results.json</code>
3D command centre draw calls (after merge)	343	in-browser measurement
Test suite size	68	<code>pytest --collect-only</code>
Feature count	52	<code>train.py:ALL_FEATURES</code>
Decision entries recorded	18	<code>grep -c '^## D-' DECISIONS.md</code>

§32. What to say when your mentor asks the hard question

"How do you know you didn't overfit the 2022 test year through your own iteration?"

The honest answer: *"I looked at the 2022 test set once, at final evaluation, after freezing the model on 2016–2020 with hyperparameters chosen on 2021. The test set was never used for feature selection or hyperparameter search. But you're right that any published number could in principle be the result of many hidden trials. The strongest defence I can offer is (a) our drift monitor shows 2022 is a genuinely atypical year — PSI 2.64 on upper-level wind shear — so the model was evaluated under distribution shift, not on an easy in-distribution slice; and (b) every hyperparameter and threshold choice is in the code with a `TRAIN_YEARS` guard. We'd need to run rolling cross-validation for a definitive answer, which is item #1 on §29."*

Do not lie about this. It is the most sophisticated question a mentor can ask, and the honest answer earns more respect than a confident false one.

Part IV — Data Policy

This project uses five datasets. **None of the raw third-party data is redistributed here** — it is external data under its own licence, and this repo pulls it straight from the primary sources with `scripts/fetch_*.py`. What is committed is (a) our own derived artifacts, small enough to live in git, and (b) a precomputed results store shipped as a GitHub Release asset.

The result: you can clone the repo and **run the tests and the offline dashboard immediately**, without downloading a single byte of raw NWP data.

1. What is committed directly in the repo (~42 MB)

These are **our derived products**, not third-party data. Safe to redistribute, small enough for git, and they make the repo runnable.

Path	Size	What it is
<code>data/processed/dataset.parquet</code>	34 MB	The full labelled modelling frame: 279,650 rows × 93 columns (subdivision × init × lead, with the bust label, all 52 features, and the split tag). This is the single file <code>train_model.py</code> needs.
<code>data/interim/imd_subdivisions.gpkg</code>	6 MB	The 34 modelled IMD subdivision polygons (built as unions of 641 Census-2011 districts). The API serves <code>/api/regions</code> from this.
<code>data/interim/truth_subdivision_daily.parquet</code>	180 KB	Observed subdivision-daily rainfall (area-mean), derived from the IMD gridded product.
<code>data/interim/weights_*.parquet</code>	~100 KB	Cached grid↔polygon area-overlap weights, keyed by grid geometry.
<code>data/artifacts/bust_model.joblib</code>	1.5 MB	The trained XGBoost booster + isotonic calibrator + feature list.
<code>data/artifacts/results.json</code>	7.5 KB	Held-out-year evaluation table (all baselines + model, per-lead, reliability).
<code>data/artifacts/*.csv</code>	<5 KB each	Ablation, SHAP importance, stress-test, and ENS-baseline comparison tables.
<code>data/artifacts/reference_distributions.npz</code>	134 KB	Training-year feature distributions, used by the drift monitor.

Derived-data provenance note: `dataset.parquet` and `truth_subdivision_daily.parquet` contain an observed-rainfall column derived from IMD's 0.25° gridded product (area-averaged to subdivisions, reformatted, and joined with model outputs). This is a transformative derivative work under IMD's research-use terms; attribution is in `LOGIC.md` §5. It is not a copy of the IMD product and cannot be used to reconstruct it.

2. What ships as a GitHub Release asset (~60 MB)

Asset	Size	What it is
<code>bulletins.sqlite</code>	63 MB	The precomputed bulletin store: 79,900 pre-scored (subdivision × init × lead) rows, each with the calibrated bust probability, bagged prediction interval, OOD status, SHAP reason strings, regime vector, and the baseline probability. This is what the offline dashboard serves.

It is a Release asset rather than a committed file because 60 MB of binary would bloat every clone of the git history. It is **fully regenerable** from the committed `dataset.parquet` + `bust_model.joblib` by running `scripts/generate_bulletins.py` (~100 s), so the Release is a convenience, not a dependency.

Get it (either works):

```
# One command – downloads and verifies the checksum:
PYTHONPATH=src python scripts/fetch_release_artifacts.py

# ...or regenerate it locally from the committed artifacts:
PYTHONPATH=src python scripts/generate_bulletins.py
```

SHA-256 of `bulletins.sqlite` (v0.1.0): `5cbb61350c2ba63dbcf2dc00bc1d13b9a3d2f9a70e66d8317121cf996e6f241c`

3. What is NOT in the repo and never will be (~570 MB of raw data)

This is external third-party data. We do not rehost it — we cite it and reproduce it from the primary source. Rehosting it would be a licensing grey area and would teach the wrong lesson about how research artifacts are shared.

Dataset	Local size	Primary source	Fetch with
IMD 0.25° gridded rainfall (2016–2022)	170 MB	imdpune.gov.in/cmpg/Griddata/Rainfall_25_NetCDF.html	<code>scripts/fetch_imd.py</code>
WeatherBench 2 IFS HRES forecasts	72 MB (of ~9 GB transferred)	<code>gs://weatherbench2/datasets/hres/...</code> (anonymous GCS)	<code>scripts/fetch_hres.py</code>
WeatherBench 2 ERA5 analysis	~450 MB	<code>gs://weatherbench2/datasets/era5/...</code>	<code>scripts/fetch_era5.py</code>
WeatherBench 2 IFS ENS (50-member)	360 KB (subsampled)	<code>gs://weatherbench2/datasets/ifs_ens/...</code>	<code>scripts/fetch_ens.py</code>
Census-2011 district boundaries	10 MB	github.com/datameet/maps	<code>scripts/fetch_boundaries.py</code>

Full reproduction from nothing:

```
python scripts/fetch_imd.py
python scripts/fetch_hres.py
python scripts/fetch_era5.py
python scripts/fetch_ens.py --year 2019 --every 6
python scripts/fetch_ens.py --year 2020 --every 6
python scripts/fetch_ens.py --year 2022 --every 3
python scripts/fetch_boundaries.py
# then rebuild everything:
python scripts/build_truth.py
python scripts/build_dataset.py
python scripts/train_model.py
python scripts/generate_bulletins.py
```

Total download time is dominated by the WeatherBench 2 HRES pull (~20 min) because WB2's zarr chunks span the globe (you download the world and slice to India locally — see `DECISIONS.md` D-002).

4. The three tiers, at a glance

└ committed in git (~42 MB) ────┐ our derived products: dataset.parquet, trained model, geometry, eval tables └────────────────────────────────┘	clone the repo -> tests pass, model loads, pipeline runs
└ GitHub Release asset (~60 MB) ────┐ bulletins.sqlite (precomputed dashboard) └────────────────────────────────┘	one command -> offline dashboard runs
└ never in the repo (~570 MB, external) ────┐ raw IMD / WeatherBench 2 / Census data └────────────────────────────────┘	scripts/fetch_*.py -> reproduce from source

This is the standard pattern for a reproducible-research repo: **cite the primary source, ship the reproduction script, host only your own derivatives**. It is what makes the repo credible rather than merely convenient.